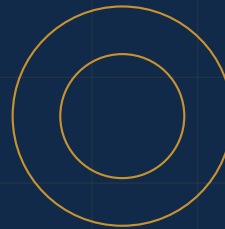


Curso avançado de Inteligência Artificial
e Machine Learning aplicado à engenharia de defesa

QUANTUM STRATEGIC AI



Inteligência Artificial e Machine Learning Avançado para Defesa

*Dos algoritmos clássicos à IA embarcada
em aplicações operacionais*

AUTOR

Cristiano Alves

Inteligência Artificial e Machine Learning Avançado para Defesa

Livro-texto dedicado à aplicação avançada de Inteligência Artificial (IA) e Machine Learning (ML) na engenharia de defesa, com carga compatível com 40 horas, organizado em quatro módulos e dez capítulos.

© 2026 Cristiano Alves.

Versões de referência das bibliotecas: `scikit-learn 1.5`, `TensorFlow 2.18`, `PyTorch 2.4`.

Ambiente principal: Google Colab. Ambiente local alternativo: Anaconda/venv com aceleração via CUDA quando disponível.

O código-fonte, os *notebooks* e os conjuntos de dados sintéticos de apoio são distribuídos sob licença MIT em repositório público. Os exemplos operacionais não contêm dados classificados; quando inspirados em sistemas reais, foram reconstruídos a partir de fontes abertas e adaptados para fins didáticos.

Esta é uma edição em desenvolvimento (versão de trabalho), destinada ao uso didático em cursos de pós-graduação, especialização das Forças Armadas e capacitação técnica em empresas da Base Industrial de Defesa. Erros, sugestões e propostas de novos estudos de caso podem ser comunicados ao autor para incorporação em edições futuras.

Composto em $\text{\LaTeX}$. Fontes: Palatino (texto) e Helvetica (títulos). Obra de continuidade ao volume *Programação com Python Aplicada à Engenharia de Defesa* (Quantum Strategic AI, 2026).

Apresentação

Este livro é a continuação natural de um percurso. O volume que o antecede, *Programação com Python Aplicada à Engenharia de Defesa*, partiu da convicção de que as tecnologias estratégicas — inteligência artificial, ciência de dados, simulação, sistemas autônomos — se materializam, todas, em software, e que a programação é, por isso, a fundação computacional do trabalho técnico. Construída essa fundação, é hora de subir um andar: passar do código que apoia a decisão à *decisão automatizada pelo modelo*; passar do gráfico que descreve um padrão à *previsão que o antecipa*; passar da análise de dados de sensores à *percepção embarcada em plataformas autônomas*. É desse salto que esta obra trata.

A literatura disponível em língua portuguesa não dá conta dessa transição. De um lado, há excelentes obras introdutórias de *machine learning* — quase todas generalistas, escritas em torno de exemplos do varejo, do setor financeiro ou da saúde —, que ensinam algoritmos sem nunca tocar nos problemas e nas restrições operacionais da defesa. De outro, há literatura jurídica especializada sobre marcos regulatórios emergentes — sistemas de armas autônomos, IA em sistemas críticos, controle humano significativo —, escrita em diálogo com o direito internacional e dissociada da formação técnica de quem efetivamente projetará esses sistemas. Faltava, em português, uma obra que articulasse, em um único volume, fundamento algorítmico, aplicação operacional e contexto regulatório, alinhada à Estratégia Nacional de Defesa e dimensionada para um curso intensivo de **40 horas**. É essa lacuna que este livro procura preencher.

A obra organiza-se em **quatro módulos e dez capítulos**, percorrendo um caminho que parte dos modelos clássicos de aprendizado de máquina, atravessa as arquiteturas profundas de percepção (visão computacional e linguagem natural, incluindo modelos de grande escala em OSINT), entra na decisão autônoma e no aprendizado por reforço aplicado a problemas táticos, e termina em três temas que respondem à pergunta mais difícil do setor: *quando* é seguro implantar um modelo, *como* é possível garantir robustez e *sob* que quadro normativo. Robustez adversarial, IA embarcada em plataformas com restrições de campo e governança de sistemas autônomos compõem o módulo final, no qual a disciplina técnica encontra a operação real.

Cada capítulo combina exposição teórica formalizada com profundidade suficiente para uso técnico, implementação executável em `scikit-learn`, TensorFlow ou PyTorch, e estudo de caso operacional extraído de aplicações reais — do *Project Maven* aos sistemas brasileiros de monitoramento de fontes abertas. Os exemplos foram ambientados em cenários de defesa nacional sempre que isso era possível sem expor dados sensíveis; quando

não foi, recorreu-se a *datasets* sintéticos cuidadosamente desenhados para reproduzir as dificuldades reais, sem reproduzir os dados reais.

Escrevi para um público que assumo qualificado e exigente: oficiais e analistas das Forças Armadas em cursos de altos estudos, engenheiros e pesquisadores da Base Industrial de Defesa, analistas de inteligência, alunos e docentes de programas de pós-graduação em Engenharia, Ciência da Computação, Defesa e Estudos Estratégicos, e profissionais do setor privado atuantes em consultoria de risco e segurança cibernética. A todos, a convicção que organizou também o volume anterior: dominar internamente esta tecnologia é uma forma concreta de *nacionalizar capacidade*. Que as ferramentas deste livro sirvam, nas suas mãos, a esse propósito.

O autor
2026

Como usar este livro

A obra cobre um curso de 40 horas, organizado em **quatro módulos** de cerca de dez horas cada, que somam dez capítulos. Cada capítulo foi pensado em **três tempos**:

Antes do encontro

uma leitura preparatória que situa os conceitos, fixa a notação e prepara o terreno matemático.

Durante o encontro

a exposição e a prática guiada, com o *notebook* executando em paralelo e os pontos de decisão técnica discutidos em sala.

Após o encontro

os exercícios de consolidação, em três níveis, para fixar o conteúdo no intervalo entre um módulo e o seguinte.

Ao longo do texto, quatro caixas reaparecem com regularidade. Vale reconhecê-las desde já:

Contexto de defesa

Conecta o conceito que está sendo estudado a uma aplicação real no domínio da defesa e segurança nacional — uma plataforma autônoma, um sistema de identificação, uma rotina de inteligência. É aqui que o algoritmo deixa de ser exercício e passa a ter consequências.

Armadilha comum

Aponta um erro frequente — de modelagem, de avaliação ou de implantação — e mostra como evitá-lo. Em *machine learning*, muitas armadilhas só se revelam fora da prova de conceito; vale a pena conhecê-las antes.

Boa prática

Recomendações de método, organização experimental e arquitetura de sistemas que separam o modelo que funciona em *notebook* daquele que se pode auditar, manter e implantar em produção.

Nota

Esclarecimentos pontuais — uma convenção de notação, um detalhe de versão de biblioteca, uma referência cruzada — que valem ser ditos mas não justificam interromper o fio da exposição.

Cada capítulo encerra com um **resumo** dos conceitos principais, uma síntese das **armadilhas comuns** e **exercícios em três níveis**: *essencial* (fixação dos conceitos do capítulo, com solução guiada no repositório), *tático* (aplicação a um problema semelhante, com escolhas técnicas a fazer) e *estratégico* (extensão criativa, frequentemente envolvendo um miniartigo de discussão técnica). Não é preciso resolver todos: os essenciais garantem o mínimo; os demais recompensam quem quer ir além.

Todo o código está disponível em um repositório público, com *notebooks* prontos para o Google Colab — um por capítulo —, executáveis no navegador, com GPU gratuita para os capítulos de *deep learning*. Recomendamos abrir o *notebook* do capítulo ao lado do texto e executar cada trecho à medida que aparece. *Machine learning* se aprende, sobretudo, refazendo os experimentos com pequenas variações: trocando o conjunto de dados, perturbando os hiperparâmetros, mudando a métrica de avaliação. É essa prática, e não a leitura, que constrói intuição.

Sumário

Apresentação	ii
Como usar este livro	iv
1 Fundamentos de Inteligência Artificial e Machine Learning	1
1.1 As três tradições da Inteligência Artificial	2
1.2 Um breve histórico, sob a ótica da defesa	3
1.3 Os três regimes de aprendizado	5
1.4 O <i>pipeline</i> de um projeto de <i>machine learning</i>	6
1.5 O caminho à frente	9
Resumo do capítulo	10
Armadilhas comuns	10
Exercícios	11
2 Modelos Clássicos de Machine Learning para Defesa	13
2.1 Regressão linear: a <i>baseline</i> de toda previsão	14
2.2 Regressão logística: da quantidade à decisão	16
2.3 Árvores de decisão e florestas aleatórias	17
2.4 Máquinas de vetores de suporte	19
2.5 Aplicação integrada: classificação de ameaças a partir de sensores	21
2.6 O caminho à frente	22
Resumo do capítulo	22
Armadilhas comuns	23
Exercícios	23
3 Redes Neurais Artificiais e Deep Learning	25
3.1 Do perceptron ao perceptron multicamadas	26
3.2 Retropropagação e o treinamento por gradiente	27
3.3 Arquiteturas profundas	30
3.4 Regularização, otimização e boas práticas de treinamento	31
3.5 Aplicação integrada: uma CNN para triagem de imagens de sensor	33
3.6 O caminho à frente	35
Resumo do capítulo	35

Armadilhas comuns	35
Exercícios	36

Fundamentos de Inteligência Artificial e Machine Learning

Objetivos do capítulo

Ao final deste capítulo, você será capaz de:

- identificar as três tradições históricas da Inteligência Artificial — a simbólica, a conexionista e a estatística — e reconhecer como elas coexistem nos sistemas contemporâneos;
- situar a evolução da IA em um eixo histórico que articula avanços científicos, ciclos de investimento e demandas operacionais da defesa;
- distinguir, com precisão técnica, os três regimes de aprendizado — supervisionado, não supervisionado e por reforço — e escolher o regime adequado a um problema operacional;
- descrever as cinco etapas do *pipeline* de um projeto de *machine learning* — dados, modelo, avaliação, implantação e operação — e reconhecer por que, em sistemas críticos de defesa, a última etapa é a mais difícil;
- articular a noção de *autonomia tecnológica* — central à Estratégia Nacional de Defesa — ao caso particular da IA, identificando os pontos da cadeia em que a dependência de modelos estrangeiros se torna risco operacional.

Este é um capítulo de orientação. Antes de tratar de qualquer modelo específico — o que faremos a partir do Capítulo 2, com os modelos clássicos de aprendizado supervisionado —, é preciso responder a quatro perguntas que dão sentido a tudo o que vem depois: *o que é, afinal, Inteligência Artificial; como ela chegou ao ponto em que está; de que formas um sistema pode aprender; e quais etapas compõem um projeto de machine learning que se pretenda implantar* — e não apenas demonstrar. As respostas estão neste capítulo. A primeira nos leva às três tradições da IA; a segunda, a um breve histórico sob a ótica da defesa; a terceira, aos três regimes de aprendizado; e a quarta, ao *pipeline* que organiza, em todos os capítulos seguintes, a passagem do problema operacional ao modelo implantado.

1.1 As três tradições da Inteligência Artificial

A expressão *Inteligência Artificial* carrega, hoje, uma carga retórica que confunde o leitor desprevenido. Na imprensa, ela é frequentemente um sinônimo de *aprendizado profundo*; em produtos comerciais, de *qualquer automação minimamente sofisticada*; em discursos prospectivos, de *sistemas que pensam como humanos*. Para um curso técnico, nada disso basta. É preciso reorganizar o vocabulário.

A IA, como disciplina científica, não é uma teoria única. Ela é, há mais de sessenta anos, a convivência tensa de **três tradições** de pesquisa, cada uma com uma resposta distinta para a pergunta *o que é, computacionalmente, um comportamento inteligente?*. Conhecê-las separadamente é o que permite reconhecer, em qualquer sistema contemporâneo, qual delas — ou que combinação — está, de fato, operando por trás da interface.

1.1.1 A tradição simbólica

A tradição mais antiga, dominante até os anos 1980, parte de uma hipótese forte: a inteligência é *manipulação de símbolos segundo regras explícitas*. Para o programa simbólico — ou *GOFAI*, *Good Old-Fashioned Artificial Intelligence*, na expressão consagrada por John Haugeland — pensar é raciocinar sobre representações estruturadas do mundo: bases de conhecimento, regras lógicas, ontologias. Um sistema especialista de diagnóstico médico, um motor de inferência jurídica, um *planner* clássico de robótica — todos pertencem a essa família. Sua força é a *explicabilidade*: cada conclusão pode ser rastreada à regra que a produziu. Sua fraqueza, descoberta na prática, é a fragilidade: a tradição simbólica funciona bem em mundos fechados, com regras codificáveis, e tropeça em *percepção* — onde os dados são ruidosos, ambíguos e de altíssima dimensionalidade.

1.1.2 A tradição conexionista

A tradição *conexionista*, nascida no fim dos anos 1950 com o perceptron de Frank Rosenblatt, parte de uma hipótese oposta: a inteligência não é manipulação de símbolos discretos, mas *propagação de ativações em uma rede de unidades elementares* — um modelo abstrato do que, fisiologicamente, faria o cérebro. Não há regras explícitas; há pesos sinápticos que são ajustados a partir de exemplos, até que a rede produza, na saída, a resposta desejada para a entrada dada. O programa conexionista atravessou dois invernos — em 1969, quando Minsky e Papert demonstraram os limites do perceptron de camada única, e em meados dos anos 1990, quando o aprendizado de redes profundas parecia intratável —, mas ressurgiu, no fim dos anos 2000, com a combinação de três ingredientes: dados em escala, hardware de aceleração (*GPUs*) e refinamentos algorítmicos do treinamento. É essa tradição que, hoje, domina a discussão pública sob o rótulo de *deep learning*.

1.1.3 A tradição estatística

A terceira tradição, mais discreta na divulgação, mas decisiva na prática, é a *estatística*. Aqui, a inteligência é compreendida como *inferência sob incerteza*: dada uma amostra, qual a distribuição que melhor a explica? Dado um novo ponto, qual a probabilidade de que ele pertença a cada classe possível? Esta tradição absorveu, ao longo das últimas três décadas, boa parte da metodologia de *machine learning* — da regressão logística aos modelos gráficos probabilísticos, das árvores de decisão aos métodos *ensemble*, das máquinas de vetores de suporte aos processos gaussianos. Sua força é o *rigor da medida de incerteza*: ela produz não apenas uma predição, mas uma estimativa de quanto se pode confiar nela. Em sistemas críticos de defesa, essa diferença não é detalhe técnico — é a

fronteira entre uma ferramenta auditável e uma caixa-preta.

1.1.4 Como as três convivem hoje

A história da disciplina é a história das três tradições se vigiando, se afastando e, periodicamente, voltando a se aproximar. O sistema de visão computacional de uma aeronave remotamente pilotada moderna usa uma rede convolucional profunda (conexionista) para detectar alvos em uma imagem; combina essas detecções com um filtro de Kalman (estatístico) para rastreamento; e submete as decisões finais a um motor de regras (simbólico) que codifica as restrições do envelope operacional autorizado. Não há, em sistemas reais, uma única tradição; há *arquiteturas híbridas* em que cada componente é escolhido pela tradição que melhor resolve o subproblema que ele endereça.

Contexto de defesa

Na engenharia de defesa, essa distinção tem consequência prática direta: cada tradição responde, de forma distinta, à exigência de *rastreabilidade* imposta pelos sistemas críticos. Um sistema simbólico se justifica explicando a regra aplicada; um modelo estatístico, exibindo a distribuição posterior e o intervalo de confiança; uma rede profunda, na sua forma bruta, *não se justifica* — daí o investimento crescente em técnicas de explicabilidade (XAI) e em camadas de verificação que envolvem o modelo connexionista em invariantes simbólicas auditáveis. Saber qual tradição opera em cada componente é, no limite, saber quem responde quando o sistema erra.

1.2 Um breve histórico, sob a ótica da defesa

Nenhum dos grandes saltos da Inteligência Artificial aconteceu fora do contexto militar. A história da disciplina é, em larga medida, a história da convivência entre a pesquisa acadêmica, a indústria de tecnologia e o financiamento de defesa — em especial o financiamento norte-americano via DARPA, o britânico via GCHQ e, mais recentemente, o chinês via planos quinquenais e o europeu via programas de defesa do bloco. Conhecer essa história ajuda a desnaturalizar a ideia, comum em apresentações comerciais, de que a IA seria um produto neutro do laboratório acadêmico. Ela não é, e nunca foi.

1.2.1 Da cibernética a Dartmouth (1940–1956)

A pré-história da IA é, simultaneamente, a pré-história da computação — e ambas nasceram da Segunda Guerra Mundial. Em Bletchley Park, a equipe de Alan Turing decifrou o tráfego cifrado da máquina Enigma com auxílio das *bombes* eletromecânicas e, mais tarde, do computador eletrônico Colossus. Do outro lado do Atlântico, o projeto ENIAC processava tabelas balísticas para o Exército norte-americano. Ainda durante a guerra, Norbert Wiener formulou as bases da *cibernética* — a ciência dos sistemas de controle com realimentação —, trabalho que partiu, originalmente, do problema de mira automática contra aeronaves inimigas. O nascimento da IA, em 1956, na conferência de verão de Dartmouth, ocorre, portanto, em um terreno já pavimentado por uma década de financiamento militar e por uma agenda intelectual em que as máquinas eram pensadas, desde o início, como agentes de decisão sob incerteza.

1.2.2 O primeiro entusiasmo e o primeiro inverno (1956–1980)

Os primeiros vinte anos da disciplina foram dominados pela tradição simbólica e por expectativas exageradas. *General Problem Solver*, *SHRDLU*, sistemas de tradução automática — tudo parecia ao alcance dentro de uma geração. Em 1969, Minsky e Papert publicaram *Perceptrons*, demonstrando os limites formais das redes neurais de camada única; o trabalho foi lido, à época, como condenação geral do programa conexionista. Em 1973, na Grã-Bretanha, o relatório Lighthill recomendou cortes drásticos em pesquisa de IA, considerando-a incapaz de cumprir as promessas que fizera. Sobreveio o *primeiro inverno*: redução de financiamento, fechamento de laboratórios, marginalização do tema nas universidades. O período é instrutivo: a IA não fracassou tecnicamente; ela fracassou em *calibrar expectativas* com seus financiadores — um padrão que se repetiria.

1.2.3 Sistemas especialistas e o segundo inverno (1980–1995)

A década de 1980 viu o renascimento simbólico via *sistemas especialistas* — programas que codificavam, em regras, o conhecimento de domínios técnicos estreitos. MYCIN diagnosticava infecções bacterianas; XCON configurava computadores da Digital Equipment Corporation, com economia anual estimada em dezenas de milhões de dólares. O Japão lançou o ambicioso *Fifth Generation Computer Systems Project*, e o Departamento de Defesa norte-americano financiou, via DARPA, a *Strategic Computing Initiative*, com objetivos explícitos de aplicação militar. A indústria se materializou em torno de máquinas *Lisp* dedicadas. No fim da década, contudo, ficou claro que os sistemas especialistas escalavam mal — a engenharia de conhecimento era cara, frágil e de difícil manutenção. Sobreveio o *segundo inverno*, mais curto que o primeiro, mas suficiente para que a expressão *Inteligência Artificial* saísse, por uma década, dos planos de negócio.

1.2.4 Estatística, dados e maturação (1995–2012)

A retomada veio com troca de paradigma: a tradição estatística, fortalecida pela crescente disponibilidade de dados digitalizados e pelo avanço da capacidade computacional, oferecia algo que os sistemas especialistas não tinham — *robustez sob incerteza*. *Máquinas de vetores de suporte*, *boosting*, *random forests*, modelos gráficos probabilísticos: um arsenal técnico maduro, com fundamentação teórica sólida, alimentou aplicações de larga escala em busca na *web*, reconhecimento de fala, filtragem de *spam*. Nessa fase, a IA deixou de ser disciplina autônoma para se dissolver em *machine learning* — termo que, até hoje, descreve melhor o que efetivamente se faz na prática.

1.2.5 A revolução do aprendizado profundo (2012–)

O ponto de inflexão é datável com precisão: setembro de 2012, competição ImageNet. A rede convolucional *AlexNet*, de Krizhevsky, Sutskever e Hinton, reduziu o erro de classificação à metade do que o segundo colocado obtivera, mostrando que arquiteturas profundas, treinadas em GPUs, sobre dados em escala, superavam toda a engenharia de *features* cuidadosamente construída pela tradição estatística clássica. A década seguinte foi de avanços contínuos: redes residuais, mecanismos de atenção, arquiteturas *Transformer*, modelos de linguagem de grande escala, modelos multimodais. A escala dos modelos cresceu de milhões para centenas de bilhões de parâmetros; o custo computacional do treinamento, de horas em uma workstation para meses em centros de dados dedicados. Foi nesse período, também, que a IA voltou a ter agenda de defesa explícita — em especial no *Project Maven* (2017), do Departamento de Defesa norte-americano, em que técnicas de

visão computacional treinadas em dados de aeronaves remotamente pilotadas levantaram, simultaneamente, questões operacionais e éticas que voltarão a aparecer ao longo deste livro.

1.2.6 A IA na defesa: *driver* e *consumidor*

Convém, antes de prosseguir, dar nome a uma estrutura recorrente. A defesa atua, ao mesmo tempo, como *driver* e *consumidor* da IA. Como *driver*, porque seu financiamento direcionou — e direciona — agendas de pesquisa que talvez não emergissem do mercado civil: visão computacional em imagens SAR, processamento de sinais em ambientes adversariais, aprendizado por reforço em simulação de combate, robustez sob ataque deliberado. Como *consumidor*, porque adota — frequentemente com atraso, por exigências de certificação — modelos e arquiteturas que se consolidaram primeiro no mundo civil. Essa dupla condição produz o que talvez seja o traço mais característico da IA aplicada à defesa contemporânea: ela vive no *descompasso* entre o ritmo da pesquisa, o ritmo dos requisitos operacionais e o ritmo, mais lento, da regulação. Boa parte das tensões discutidas no Capítulo 10 deste livro nasce desse descompasso.

Contexto de defesa

O ponto não é trivial para o leitor brasileiro. A Estratégia Nacional de Defesa, em sua ênfase na *nacionalização de tecnologias sensíveis*, identifica a IA como uma das áreas em que a dependência externa se torna risco operacional. Adotar acriticamente modelos pré-treinados em *datasets* estrangeiros, em que a Amazônia, a fronteira amazônica ou a Amazônia Azul aparecem mal ou não aparecem, é importar um *viés geográfico* no produto final. Construir soberanamente significa, em IA, controlar a cadeia que vai dos dados de coleta ao modelo implantado.

1.3 Os três regimes de aprendizado

Tratamos, na seção anterior, da IA *como disciplina*; trataremos, nesta, de uma decomposição interna do *machine learning* — sua tradição dominante hoje. Aqui interessa menos a história e mais uma distinção operacional: *de que forma* um modelo aprende a partir dos dados que lhe são apresentados. A literatura técnica reconhece três regimes principais. A escolha entre eles é, talvez, a primeira decisão estratégica de qualquer projeto operacional — antes de qualquer algoritmo específico, antes de qualquer biblioteca.

1.3.1 Aprendizado supervisionado

No regime *supervisionado*, dispomos de um conjunto de dados rotulados: para cada entrada $\mathbf{x}_i$, conhecemos a saída desejada y_i . O objetivo é ajustar uma função $\hat{f}$ tal que $\hat{f}(\mathbf{x}) \approx y$ para todo par $(\mathbf{x}, y)$ do conjunto — e, mais importante, para pares novos, não observados durante o treinamento. Quando y é categórico, o problema é de *classificação*; quando contínuo, de *regressão*.

Aplicações operacionais típicas em defesa: a *classificação automática de assinaturas acústicas* de plataformas navais (cada espectro recebe a etiqueta do tipo de embarcação que o produziu); a *detecção de alvos* em imagens de aeronaves remotamente pilotadas (cada janela da imagem é rotulada como *contém alvo* ou *não contém alvo*); a *previsão de falha em componente* de uma plataforma militar (cada histórico de operação é rotulado com o tempo até a próxima falha registrada). Em todos os casos, o gargalo prático é a

rotulagem: produzir os pares (x_i, y_i) exige especialistas — sonaristas, analistas de imagem, engenheiros de manutenção — cujo tempo é a verdadeira matéria-prima do projeto.

1.3.2 Aprendizado não supervisionado

No regime *não supervisionado*, dispomos apenas das entradas x_i : não há rótulo. O objetivo passa a ser descobrir, a partir dos próprios dados, *estrutura* — agrupamentos, eixos de variação relevantes, densidades, anomalias. Pertencem a esse regime os métodos de *clustering* (*k-means*, agrupamento hierárquico, *DBSCAN*), a *redução de dimensionalidade* (análise de componentes principais, *t-SNE*, *UMAP*) e a *detecção de anomalias*.

Em aplicações operacionais, o aprendizado não supervisionado costuma ocupar a fase de *exploração* de um novo conjunto de dados — antes que se invista no esforço de rotulagem — e a fase de *vigilância de comportamento*, em que o que interessa é detectar o que *destoa* do padrão histórico. A detecção de tráfego de rede atípico, a identificação de movimentos suspeitos em fronteira a partir de séries temporais de sensores, o reconhecimento de publicações coordenadas em redes sociais — todos esses problemas se beneficiam, ao menos como ponto de partida, de técnicas não supervisionadas.

1.3.3 Aprendizado por reforço

O terceiro regime, *aprendizado por reforço*, é qualitativamente diferente dos dois anteriores. Não há um conjunto de dados pré-existente; há um *agente* que interage com um *ambiente*, escolhendo *ações* e recebendo, em troca, *recompensas* numéricas. O objetivo é descobrir uma *política* — uma função do estado observado à ação escolhida — que maximize a recompensa acumulada ao longo do tempo. Os algoritmos clássicos (*Q-learning*, *SARSA*) e os modernos profundos (*DQN*, *PPO*, *A3C*) compartilham essa formulação.

O aprendizado por reforço é, conceitualmente, o regime mais próximo do que se entende intuitivamente por *decisão autônoma*: um sistema que aprende, por tentativa e erro em simulação, a operar bem em um ambiente. Suas aplicações de defesa são, simultaneamente, as mais promissoras e as mais delicadas: planejamento de trajetórias de plataformas autônomas, alocação dinâmica de recursos em operações conjuntas, controle de enxames. Dedicaremos a essas aplicações o inteiro Capítulo 7 — e a discussão ética que delas decorre, o Capítulo 10.

Boa prática

A primeira decisão técnica de qualquer projeto operacional não é a escolha do algoritmo, mas a escolha do *regime de aprendizado*. Frequentemente, uma equipe se compromete cedo com aprendizado supervisionado — porque é o regime mais ensinado — e descobre tarde que não tem rótulos confiáveis, ou que os rótulos disponíveis foram gerados por um processo enviesado que o modelo aprenderá a reproduzir. Reservar, na fase inicial do projeto, um workshop dedicado a essa decisão estratégica — *de fato, este problema admite rótulos? Em que escala? A que custo? Com que viés sistemático?* — economiza meses no fim.

1.4 O pipeline de um projeto de machine learning

Um modelo de *machine learning* é um produto, e produtos se constroem em *etapas*. O *pipeline* de cinco etapas que apresentamos a seguir não é, em si, original — variantes dele

aparecem com nomes diferentes (*CRISP-DM*, *CRISP-ML(Q)*, ciclo de vida *MLOps*) — mas é, talvez, o instrumento conceitual mais útil para organizar todo o trabalho dos capítulos seguintes. Cada capítulo posterior, ao tratar de um novo modelo, revisitará uma ou mais dessas etapas; vale, por isso, fixá-las desde já.

1.4.1 Dados

A primeira etapa é a mais subestimada. *Dados* significa identificar as fontes — sensores, registros operacionais, fontes abertas, dados sintéticos —; estabelecer pipelines de *ingestão*; resolver questões de *qualidade* (*outliers*, *missing*, inconsistências); aplicar *transformações* (normalização, codificação, engenharia de *features*); e — em supervisionado — produzir os *rótulos*. Estimativas amplamente citadas na literatura industrial apontam que essa etapa consome entre 60% e 80% do tempo total de um projeto de *machine learning*; em projetos de defesa, com exigências adicionais de segurança, classificação e auditabilidade da cadeia de custódia dos dados, o percentual tende a ser ainda maior.

1.4.2 Modelo

A segunda etapa — o *modelo* — é a que recebe atenção desproporcional na literatura didática. Ela inclui a escolha da família do modelo (regressão logística, *random forest*, rede neural, *Transformer*), a definição da arquitetura específica, a escolha do procedimento de otimização (*SGD*, *Adam*, taxas de aprendizado adaptativas), a definição dos hiperparâmetros e a estratégia de busca por eles. É a etapa em que os *notebooks* se concentram, em que os blogs técnicos se demoram, e em que, com frequência, projetos perdem tempo otimizando algo que mal supera, em operação, um modelo mais simples bem implantado.

1.4.3 Avaliação

A terceira etapa — *avaliação* — é o ponto em que os bons projetos se separam dos demais. Avaliar um modelo significa escolher *métricas* adequadas ao problema (não basta acurácia em dados desbalanceados; em detecção de ameaças raras, falsos negativos têm peso operacional incomparavelmente maior que falsos positivos); definir uma *estratégia de validação* que respeite a estrutura dos dados (*cross-validation*, validação temporal, validação por grupo); construir, sempre que possível, uma *baseline* honesta contra a qual o modelo proposto seja comparado; e analisar *erros sistemáticos* — em quais subgrupos o modelo erra mais? Que viés geográfico, temporal ou de aquisição estamos importando?

Armadilha comum

A armadilha mais frequente do iniciante — e, muito mais perigosa, a do projeto operacional — é a *fuga de informação* (*data leakage*): algum sinal sobre a resposta vaza, durante a preparação dos dados, para o conjunto de treinamento. O modelo aprende a explorar esse sinal, atinge métricas excelentes no conjunto de validação e *falha em produção*, onde o sinal vazado não existe. Em séries temporais, a forma típica é usar informação futura para prever o passado; em dados clínicos, é incluir, entre as *features*, alguma variável que só se conhece após o desfecho que se quer prever. Em projetos de defesa, é particularmente comum quando se cruzam bases de dados produzidas por sistemas distintos. Antes de comemorar uma métrica alta, vale a pergunta: *como, exatamente, este modelo aprendeu a acertar tanto?*

1.4.4 Implantação

A quarta etapa — *implantação* — é a que separa, com clareza, o exercício acadêmico do produto operacional. Implantar um modelo significa empacotá-lo de forma reproduzível (versões fixadas das bibliotecas, hardware especificado), construir as interfaces pelas quais o sistema o invoca, garantir latência e *throughput* compatíveis com a operação, e fornecer *telemetria* — observabilidade — que permita monitorar o desempenho ao longo do tempo. Em sistemas embarcados — VANTs, plataformas autônomas, sistemas táticos — junta-se a restrição de *recursos*: energia, memória, latência de inferência. É a esses problemas que dedicaremos o Capítulo 9.

1.4.5 Operação: o que vem depois da entrega

Acrescentamos, ao *pipeline* clássico de quatro etapas, uma quinta: *operação*. Um modelo implantado não está pronto: ele convive com *drift* dos dados (a distribuição muda lentamente — a frota envelhece, o adversário muda táticas, o sensor se descalibra), com *ataques deliberados* (exemplos adversariais, envenenamento de dados de retreinamento), com *requisitos regulatórios* que evoluem. Manter um modelo significa re-treinar, re-validar, re-implantar — e documentar, a cada ciclo, o que mudou. É essa fase de operação, e não a fase de modelagem, que dá origem ao campo emergente conhecido como *MLOps*, e é nela que residem as questões mais difíceis de *governança* discutidas no Capítulo 10.

1.4.6 Da prova de conceito ao sistema crítico

Vale insistir num ponto que organiza, em última instância, toda esta obra: em defesa, *nenhum modelo é entregue como prova de conceito*. Mesmo ferramentas de apoio à análise — que não tomam decisões letais, que apenas sugerem ao analista uma hipótese — precisam ser auditáveis, reproduzíveis e robustas a entradas mal-intencionadas. A passagem da prova de conceito ao sistema crítico é a passagem que custa anos, e que a maior parte dos cursos introdutórios de *machine learning* simplesmente não trata. Este livro, em particular nos Módulos III e IV, faz dela o tema central.

Nota

A discussão completa do *pipeline* virá nos capítulos seguintes — em particular, no Capítulo 2 trataremos com profundidade da fase de *dados* e *avaliação* para modelos clássicos, no Capítulo 9 da fase de *implantação* sob restrição embarcada, e no Capítulo 10 das exigências de auditabilidade e governança. Não é necessário, neste momento, dominar cada etapa; é necessário ter, em mente, o esqueleto que organiza o trabalho técnico do livro inteiro.

Para encerrar este capítulo de fundamentos, vale ver, em código, o esqueleto mínimo do *pipeline* que acabamos de descrever. O exemplo, propositadamente simples, usa um conjunto sintético e o classificador mais elementar possível; seu valor está em *nomear*, com nomes de variáveis, as cinco etapas que voltarão, com nuance crescente, em todos os capítulos seguintes.

Listagem 1.1: Esqueleto mínimo de um *pipeline* de classificação.

```
1 # Pipeline mínimo de machine learning supervisionado.
2 from sklearn.datasets import make_classification
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
```



```

5 from sklearn.linear_model import LogisticRegression
6 from sklearn.metrics import classification_report
7
8 # 1. Dados -----
9 X, y = make_classification(n_samples=1000, n_features=8, random_state=0)
10 X_treino, X_teste, y_treino, y_teste = train_test_split(
11     X, y, test_size=0.25, random_state=0, stratify=y
12 )
13
14 # 2. Modelo (com transformacao de dados acoplada) -----
15 escalador = StandardScaler()
16 X_treino_n = escalador.fit_transform(X_treino)
17 X_teste_n = escalador.transform(X_teste) # mesma transformacao do treino
18
19 modelo = LogisticRegression(max_iter=1000, random_state=0)
20 modelo.fit(X_treino_n, y_treino)
21
22 # 3. Avaliacao -----
23 y_predito = modelo.predict(X_teste_n)
24 print(classification_report(y_teste, y_predito, digits=3))
25
26 # 4. Implantacao (esboco) -----
27 # import joblib
28 # joblib.dump({"escalador": escalador, "modelo": modelo}, "modelo_v1.joblib")
29
30 # 5. Operacao (esboco) -----
31 # - monitorar a distribuicao das entradas em producao
32 # - recolher feedback de erros para retreinamento periodico
33 # - re-validar antes de re-implantar

```

A saída esperada, para a execução do trecho acima com a semente `random_state=0`, é um relatório de classificação com *precisão*, *revocação* e *F1-score* por classe; as métricas exatas variam ligeiramente entre versões da biblioteca, mas o desempenho do classificador deve situar-se na faixa de 85% a 90% de F1. Mais importante que o número, no entanto, é a estrutura: as cinco etapas estão lá, nomeadas, com a lógica que revisitaremos em todos os modelos seguintes.

1.5 O caminho à frente

A partir deste capítulo, percorremos o livro em quatro módulos, dimensionados para cerca de dez horas de aula cada.

O Módulo I — *Fundamentos e Modelos Clássicos* — encerra-se nos Capítulos 2 e 3, em que tratamos dos modelos supervisionados clássicos (regressão linear e logística, árvores de decisão e florestas, máquinas de vetores de suporte) e do salto para o *deep learning* (perceptron, MLP, retropropagação, primeiras arquiteturas profundas).

O Módulo II — *Percepção: Visão e Linguagem* — atravessa os Capítulos 4, 5 e 6: visão computacional aplicada a alvos operacionais (YOLO, *Faster R-CNN*, imagens SAR); processamento de linguagem natural para inteligência; e modelos de linguagem de grande escala aplicados a OSINT.

O Módulo III — *Decisão Autônoma e Aprendizado por Reforço* — cobre os Capítulos 7 e 8: aprendizado por reforço profundo para decisão tática e robustez adversarial em IA aplicada a sistemas críticos.

O Módulo IV — *Confiabilidade, Operação e Governança* — fecha o livro com os Capítulos 9 e 10: IA embarcada e computação em tempo real, e o quadro ético e regulatório dos

sistemas autônomos, do princípio do controle humano significativo à posição brasileira em foros internacionais.

A ordem foi escolhida com cuidado. Os fundamentos do Módulo I dão o vocabulário; a percepção do Módulo II constrói os blocos mais usados na prática; a decisão autônoma do Módulo III combina percepção e ação; e a governança do Módulo IV fecha o ciclo — porque, em defesa, nenhum modelo se discute, no fim, sem o contexto normativo em que opera.

Resumo do capítulo

- A Inteligência Artificial não é uma teoria única, mas a convivência de **três tradições** — *simbólica, connexionista e estatística* —, cada uma com sua resposta para o que constitui comportamento inteligente. Sistemas reais combinam as três em arquiteturas híbridas.
- A história da disciplina, lida sob a ótica da defesa, é uma sequência de *entusiasmos e invernos* produzida, em larga medida, pelo descompasso entre promessas e calibração de expectativas. O ponto de inflexão contemporâneo é a *revolução do aprendizado profundo* (2012–), com aplicações de defesa explícitas a partir do *Project Maven* (2017).
- Em *machine learning*, três **regimes de aprendizado** organizam a prática: *supervisionado* (com rótulos), *não supervisionado* (sem rótulos, descobrindo estrutura) e *por reforço* (interação com ambiente via recompensas). A escolha do regime adequado ao problema operacional é a primeira decisão estratégica de qualquer projeto.
- Um projeto de *machine learning* é organizado em um **pipeline de cinco etapas** — *dados, modelo, avaliação, implantação, operação*. Em sistemas críticos de defesa, a passagem da *prova de conceito* ao *sistema implantado* é o problema central, e não o desempenho do modelo no *notebook*.
- A *autonomia tecnológica* — central à Estratégia Nacional de Defesa — se traduz, em IA, em controlar a cadeia que vai dos *dados de coleta* ao *modelo implantado*. Adotar acriticamente modelos estrangeiros importa, junto, seus vieses.

Armadilhas comuns

- **Confundir IA com *deep learning*.** A IA é mais ampla; o *deep learning* é, hoje, a tradição mais visível, mas não é a única. Em muitas aplicações operacionais, modelos clássicos — estatísticos ou baseados em regras — são mais auditáveis, mais robustos e mais adequados.
- **Subestimar a fase de dados.** A literatura didática se demora no modelo; em projetos reais, o gargalo está quase sempre na qualidade, rotulagem e curadoria dos dados. Investimento desproporcional em modelagem, com pouco em dados, é o padrão de quem está começando.
- **Tratar a avaliação como uma única métrica.** Acurácia agregada em conjuntos desbalanceados engana. Em detecção de ameaças, em diagnóstico, em sistemas críticos, é preciso métrica diferenciada por tipo de erro e análise por subgrupo. O custo operacional de

cada tipo de erro é específico do problema.

- **Confundir *notebook* com sistema implantado.** O modelo que funciona em *notebook* ainda está longe do que opera em produção. Versões fixadas, latência, telemetria, monitoramento de *drift*, ciclo de retreinamento — tudo isso é trabalho que se torna visível só na transição para operação.
- **Importar viés geográfico via modelos pré-treinados.** Adotar modelos pré-treinados em *datasets* estrangeiros, sem avaliar seu desempenho em dados nacionais, é importar vieses sistemáticos — em particular nas tarefas de percepção, em que a representação da geografia, da fauna, da vegetação e das próprias plataformas é culturalmente específica.

Exercícios

Essencial — fixação, com solução guiada no repositório

Exercício 1.1 Em até uma página, redija, com suas próprias palavras, uma explicação que contraste as três tradições da IA — simbólica, conexcionista e estatística — indicando, para cada uma, (a) sua tese central sobre o que é comportamento inteligente, (b) um exemplo de aplicação em que ela é, ainda hoje, a melhor escolha técnica e (c) uma limitação operacional típica.

Exercício 1.2 Identifique, em uma reportagem técnica recente sobre IA em defesa (qualquer periódico de circulação reconhecida), qual das três tradições parece estar operando no sistema descrito. Em seguida, formule duas perguntas que você faria à equipe responsável para verificar sua hipótese.

Exercício 1.3 Execute, em um *notebook* do Google Colab, o esqueleto do *pipeline* da Listagem 1.1. Verifique se o relatório de classificação corresponde ao esperado. Em seguida, altere o parâmetro `test_size` para 0,50 e relate, em duas linhas, o efeito observado sobre as métricas.

Tático — aplicação a um problema semelhante

Exercício 1.4 Considere o problema operacional de *detecção de embarcações suspeitas em rotas comerciais a partir de dados AIS*. Para cada uma das quatro etapas iniciais do *pipeline* de *machine learning* (dados, modelo, avaliação, implantação), liste três decisões técnicas concretas que precisariam ser tomadas, e identifique, em cada decisão, o principal risco operacional associado.

Exercício 1.5 Discuta, em um breve texto técnico, sob qual dos três regimes de aprendizado você atacaria os seguintes problemas, justificando: (a) prever o consumo de combustível de uma frota de veículos militares a partir do perfil de missão; (b) descobrir grupos de comportamento atípico em metadados de tráfego de uma rede tática; (c) treinar um agente para conduzir um VANT em ambiente simulado com obstáculos móveis.

Exercício 1.6 Modifique a Listagem 1.1 substituindo o classificador `LogisticRegression` por `RandomForestClassifier` (de `sklearn.ensemble`). Mantenha as demais etapas. Compare os relatórios de classificação dos dois modelos. Em uma linha, formule uma hipótese sobre *por que* a diferença ocorre — e, na linha seguinte, descreva um experimento que permitiria testá-la.

Estratégico — extensão criativa

Exercício 1.7 Leia o trecho da Estratégia Nacional de Defesa que trata da nacionalização de tecnologias sensíveis. Identifique, ao longo do texto, as menções explícitas ou implícitas

à *Inteligência Artificial* ou a tecnologias correlatas e, em um breve ensaio (duas páginas), discuta como o *pipeline* de *machine learning* apresentado neste capítulo se articula ao princípio da soberania tecnológica — em que pontos da cadeia, especificamente, a dependência externa se torna risco operacional.

Exercício 1.8 Esboce, em um diagrama, a arquitetura híbrida de um sistema operacional de sua escolha (por exemplo, um sistema de vigilância marítima), explicitando qual tradição da IA opera em cada componente e justificando as escolhas. Indique, no mesmo diagrama, em que pontos as interfaces entre componentes representam risco — de *drift* de dados, de ataque adversarial, de incompatibilidade de garantias.

Exercício 1.9 Escolha um dos *datasets* brasileiros de domínio público mencionados em relatórios oficiais (por exemplo, os disponibilizados pelo INPE, pelo IBGE ou por iniciativas de governo aberto). Avalie, em duas páginas, se ele seria adequado como base para algum dos problemas operacionais discutidos neste capítulo. Identifique, em particular, os vieses sistemáticos, a cobertura geográfica e os limites de uso documental que precisariam ser reconhecidos.

Modelos Clássicos de Machine Learning para Defesa

Objetivos do capítulo

Ao final deste capítulo, você será capaz de:

- formular e treinar modelos de *regressão linear* e *regressão logística*, distinguindo com clareza quando o problema é de previsão de quantidade e quando é de decisão entre classes;
- explicar o funcionamento de *árvores de decisão* e de *florestas aleatórias* (*random forests*), e extrair delas medidas de importância de variáveis úteis à auditoria;
- compreender a formulação das *máquinas de vetores de suporte* (SVM), o conceito de margem máxima e o papel dos *kernels* na separação de dados não lineares;
- escolher métricas de avaliação adequadas a problemas de classificação com *classes desbalanceadas* — situação típica na detecção de ameaças — e interpretar a matriz de confusão em termos de custo operacional;
- construir, em *scikit-learn*, um *pipeline* comparativo que confronte vários modelos clássicos sobre um mesmo problema de classificação a partir de dados de sensores.

No Capítulo 1, organizamos o vocabulário e fixamos o esqueleto do trabalho: as três tradições da IA, os três regimes de aprendizado e o *pipeline* de cinco etapas. Neste capítulo, descemos do panorama à prática e tratamos da família de modelos com que quase todo projeto operacional *deveria* começar — os **modelos clássicos** de *machine learning* supervisionado. São eles que, na maioria dos problemas reais de defesa, formam a *baseline* honesta contra a qual qualquer arquitetura profunda terá de se justificar; e são eles, com frequência, a escolha final, por serem mais auditáveis, mais robustos com poucos dados e mais fáceis de manter em produção.

A escolha de tratá-los antes do *deep learning* — assunto do Capítulo 3 — não é cronológica nem hierárquica; é metodológica. Em sistemas críticos, a pergunta certa não é

qual o modelo mais sofisticado que consigo treinar?, mas qual o modelo mais simples que resolve o problema com garantias suficientes?. Este capítulo é, em larga medida, uma defesa dessa pergunta.

2.1 Regressão linear: a *baseline* de toda previsão

A *regressão linear* é o modelo mais simples do aprendizado supervisionado, e é também — não por acaso — o ponto de partida correto para qualquer problema de *regressão* (previsão de uma quantidade contínua). Sua simplicidade não é defeito: é uma virtude. Um modelo que se entende completamente, cujos coeficientes têm interpretação direta e cujo comportamento se prevê fora dos dados de treino é, em muitos contextos operacionais, preferível a uma alternativa opaca de desempenho marginalmente superior.

2.1.1 Formulação

Dado um vetor de entrada $\mathbf{x} = (x_1, x_2, \dots, x_d)$ com d atributos, a regressão linear modela a saída $\hat{y}$ como uma combinação linear das entradas:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = \mathbf{w}^\top \mathbf{x} + b,$$

em que $\mathbf{w} = (w_1, \dots, w_d)$ é o vetor de *pesos* (ou coeficientes) e b é o *viés* (*intercept*). Treinar o modelo significa encontrar os valores de $\mathbf{w}$ e b que minimizam o erro *quadrático médio* (*mean squared error*, MSE) sobre os n exemplos do conjunto de treinamento:

$$\text{MSE}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2.$$

Para a regressão linear, esse problema de minimização tem *solução fechada* (as chamadas equações normais), o que a torna excepcionalmente barata de treinar e perfeitamente reproduzível: dados os mesmos dados, os mesmos coeficientes.

2.1.2 Interpretação dos coeficientes

A grande força operacional da regressão linear é a *leitura direta* de seus coeficientes. Cada w_j exprime quanto a saída prevista varia para um aumento unitário no atributo x_j , mantidos os demais constantes. Em um modelo de previsão de consumo de combustível de uma frota, por exemplo, um coeficiente positivo associado à *carga transportada* e outro à *velocidade média* têm interpretação física imediata — e podem ser confrontados com o conhecimento de engenharia para detectar erros de modelagem ou de dados.

Contexto de defesa

Em manutenção preditiva de plataformas militares — viaturas, aeronaves, motores navais —, a regressão linear é frequentemente o primeiro modelo a se testar para estimar grandezas como horas até a próxima intervenção ou consumo esperado em uma missão. Sua transparência é decisiva: quando o sistema de apoio à decisão logística recomenda antecipar a manutenção de um meio, o oficial responsável precisa poder *auditar a recomendação* — e um modelo cujos coeficientes se leem em linguagem de engenharia permite exatamente isso. Uma rede neural com desempenho 2% superior, mas inaudível, raramente compensa a perda de rastreabilidade nesse contexto.

2.1.3 Em código

A Listagem 2.1 ajusta uma regressão linear a um conjunto sintético que simula a relação entre o perfil de uma missão (distância, carga, velocidade média) e o consumo de combustível observado. O exemplo introduz o padrão que se repetirá em todo o capítulo: separar treino e teste, ajustar o modelo, prever e medir o erro.

Listagem 2.1: Regressão linear para estimativa de consumo.

```

1  # Estimativa de consumo de combustível a partir do perfil de missão.
2  import numpy as np
3  from sklearn.model_selection import train_test_split
4  from sklearn.linear_model import LinearRegression
5  from sklearn.metrics import mean_absolute_error, r2_score
6
7  rng = np.random.default_rng(0)
8  n = 400
9  distancia = rng.uniform(50, 800, n)      # km
10 carga      = rng.uniform(0.5, 8.0, n)     # toneladas
11 velocidade = rng.uniform(40, 90, n)      # km/h
12
13 # Relacao "verdadeira" + ruido (desconhecida do modelo)
14 consumo = (0.18 * distancia + 7.5 * carga + 0.9 * velocidade
15           + rng.normal(0, 12, n))        # litros
16
17 X = np.column_stack([distancia, carga, velocidade])
18 y = consumo
19
20 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)
21
22 modelo = LinearRegression()
23 modelo.fit(X_tr, y_tr)
24 y_pred = modelo.predict(X_te)
25
26 print("Coeficientes:", np.round(modelo.coef_, 3))
27 print("Intercepto: ", round(modelo.intercept_, 2))
28 print("MAE (litros):", round(mean_absolute_error(y_te, y_pred), 2))
29 print("R2:         ", round(r2_score(y_te, y_pred), 3))

```

Os coeficientes estimados devem aproximar-se dos valores que geraram os dados (0,18; 7,5; 0,9), e o coeficiente de determinação R^2 deve situar-se próximo de 0,9 — sinal de que o modelo linear captura bem a estrutura. Mais relevante que os números é o método: a regressão linear oferece, de graça, uma *baseline* interpretável que qualquer modelo mais complexo terá de superar de forma convincente para justificar sua adoção.

Boa prática

Nunca apresente um modelo complexo sem compará-lo a uma *baseline* simples. Para regressão, a *baseline* mínima é prever sempre a média do alvo; o passo seguinte é uma regressão linear. Se um modelo sofisticado não supera de modo claro essas referências, ou o problema é intrinsecamente difícil, ou — bem mais comum — há erro no pipeline. A *baseline* é, ao mesmo tempo, termo de comparação e instrumento de depuração.

2.2 Regressão logística: da quantidade à decisão

A maior parte dos problemas operacionais de defesa não pede a previsão de uma *quantidade*, mas uma *decisão entre classes*: este contato é hostil ou neutro? esta imagem contém um alvo? este tráfego de rede é legítimo ou anômalo? Para esses problemas de *classificação*, o modelo clássico fundamental é a *regressão logística* — que, a despeito do nome, é um classificador.

2.2.1 Da combinação linear à probabilidade

A ideia é elegante: parte-se da mesma combinação linear da regressão, $z = \mathbf{w}^\top \mathbf{x} + b$, mas passa-se o resultado por uma função que o comprime ao intervalo $[0, 1]$, interpretável como probabilidade. Essa função é a *sigmoide* (ou logística):

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b).$$

A saída é a probabilidade estimada de a entrada pertencer à classe positiva. A decisão final se obtém por um *limiar* (*threshold*): tipicamente, classifica-se como positivo se $P(y = 1 \mid \mathbf{x}) \geq 0,5$. Esse limiar, porém, não é sagrado — e ajustá-lo é uma das alavancas operacionais mais importantes, como veremos.

O treinamento minimiza a *entropia cruzada* (*log-loss*), que penaliza com severidade as previsões confiantes e erradas:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right], \quad \hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b).$$

2.2.2 A probabilidade como informação operacional

A regressão logística não devolve apenas um rótulo: devolve uma *probabilidade calibrada* (ou aproximadamente calibrada). Essa é uma diferença qualitativa em relação a classificadores que cospem apenas a decisão. Em um sistema de apoio à decisão, saber que um contato tem 0,55 de probabilidade de ser hostil é operacionalmente distinto de saber que tem 0,98 — ainda que ambos cruzem o limiar de 0,5. A probabilidade é, ela própria, informação para o operador.

Nota

Modelos baseados em combinação linear — regressão linear, regressão logística, SVM — são sensíveis à *escala dos atributos*. Um atributo medido em metros e outro em quilômetros entram na mesma soma ponderada com pesos de ordens de grandeza diferentes, o que prejudica a otimização. Por isso, *padronizar* os atributos (subtrair a média, dividir pelo desvio padrão) antes do treino é prática indispensável. Modelos baseados em árvores, que veremos a seguir, são imunes a esse problema — uma de suas vantagens práticas.

2.2.3 Classes desbalanceadas e o custo do erro

Aqui surge a primeira armadilha séria da classificação operacional. Em defesa, as classes de interesse são, com frequência, *raras*: a vasta maioria dos contatos é neutra, a vasta maioria do tráfego é legítima, a vasta maioria das imagens não contém alvo. Um classificador que simplesmente respondesse “*classe majoritária*” a tudo atingiria *acurácia* altíssima — e seria operacionalmente inútil, pois nunca detectaria o que importa.

A Listagem 2.2 treina uma regressão logística sobre um conjunto desbalanceado que simula a triagem de contatos, com apenas 12% de exemplos hostis, e exibe a matriz de confusão e o relatório por classe — não a acurácia agregada.

Listagem 2.2: Regressão logística em problema desbalanceado de triagem.

```

1 # Triagem de contatos com classes desbalanceadas (12% hostis).
2 from sklearn.datasets import make_classification
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.metrics import confusion_matrix, classification_report
7
8 X, y = make_classification(
9     n_samples=2000, n_features=10, n_informative=5,
10    weights=[0.88, 0.12], random_state=0 # 12% da classe positiva (hostil)
11 )
12 X_tr, X_te, y_tr, y_te = train_test_split(
13     X, y, test_size=0.25, random_state=0, stratify=y
14 )
15
16 escalador = StandardScaler().fit(X_tr)
17 X_tr_n, X_te_n = escalador.transform(X_tr), escalador.transform(X_te)
18
19 # class_weight="balanced" compensa o desbalanceamento das classes
20 modelo = LogisticRegression(max_iter=1000, class_weight="balanced")
21 modelo.fit(X_tr_n, y_tr)
22 y_pred = modelo.predict(X_te_n)
23
24 print("Matriz de confusao:\n", confusion_matrix(y_te, y_pred))
25 print(classification_report(y_te, y_pred, digits=3,
26     target_names=["neutro", "hostil"]))

```

O parâmetro `class_weight="balanced"` instrui o modelo a ponderar mais os erros sobre a classe rara, deslocando o equilíbrio em favor da *revocação* (*recall*) da classe hostil — isto é, da fração de ameaças efetivamente detectadas. Esse deslocamento aumenta os falsos positivos (alarmes sobre contatos neutros), e essa é exatamente a troca que o projetista precisa calibrar conscientemente, em diálogo com o operador.

Armadilha comum

A acurácia agregada é a métrica mais enganosa em problemas de classe rara. Com 12% de exemplos hostis, um classificador que nunca dispara o alarme acerta 88% das vezes — e é inútil. *Nunca* avalie um problema desbalanceado por acurácia. Use *precisão* e *revocação* por classe, o *F1-score*, a matriz de confusão e, quando o limiar for ajustável, a curva *precision-recall*. Em defesa, o ponto é ainda mais agudo: falsos negativos (ameaças não detectadas) e falsos positivos (alarmes falsos) têm custos operacionais radicalmente diferentes, e é esse custo — não a acurácia — que deve guiar a escolha do limiar.

2.3 Árvores de decisão e florestas aleatórias

Os modelos lineares supõem que a relação entre atributos e saída é, em alguma escala, linear. Quando essa hipótese falha — quando há interações complexas entre atributos, limiares abruptos, regiões de decisão irregulares —, uma família diferente se mostra mais

adequada: os modelos baseados em *árvores*.

2.3.1 A árvore de decisão

Uma *árvore de decisão* classifica uma entrada por uma sequência de perguntas binárias sobre seus atributos: *a velocidade é maior que 40 nós?*; em caso afirmativo, *a assinatura acústica está na banda X?*; e assim por diante, até uma folha que atribui a classe. Cada nó interno escolhe o atributo e o limiar que melhor *separam* as classes, segundo uma medida de *impureza*. A mais comum é o *índice de Gini*:

$$G = 1 - \sum_{k=1}^K p_k^2,$$

em que p_k é a proporção de exemplos da classe k no nó. Gini é zero quando o nó é puro (uma só classe) e máximo quando as classes se distribuem por igual. O algoritmo escolhe, a cada divisão, o corte que mais reduz a impureza ponderada dos nós-filhos.

A virtude das árvores é dupla: elas capturam relações não lineares e interações sem que se precise especificá-las, e produzem um modelo *legível* — o caminho da raiz à folha é uma regra explícita, auditável, próxima da forma como um analista descreveria seu próprio raciocínio. Sua fraqueza é a tendência ao *sobreajuste* (*overfitting*): uma árvore profunda o bastante memoriza o ruído do treino, decorando os dados em vez de aprender o padrão.

2.3.2 Da árvore à floresta

A floresta aleatória (*random forest*) corrige a fragilidade da árvore isolada por uma ideia simples e poderosa: em vez de uma árvore, treinam-se *muitas* — centenas —, cada uma sobre uma amostra aleatória dos dados (com reposição, técnica chamada *bagging*) e considerando, em cada divisão, apenas um subconjunto aleatório dos atributos. A predição final é o *voto majoritário* das árvores. O efeito é estatístico: erros individuais, descorrelacionados entre árvores, tendem a se cancelar no agregado, e o conjunto generaliza muito melhor que qualquer árvore isolada.

Contexto de defesa

A floresta aleatória é, talvez, o modelo clássico mais usado em problemas operacionais de classificação a partir de dados de sensores. Para classificação de assinaturas — acústicas, radar, eletro-ópticas —, ela combina três virtudes raras de aparecerem juntas: desempenho robusto sem ajuste fino, tolerância a atributos de escalas distintas e, sobretudo, uma medida de *importância de atributos* que indica *quais* características do sinal mais pesaram na decisão. Essa última propriedade é ouro para a auditabilidade: permite ao analista verificar se o modelo se apoia em características fisicamente plausíveis do sinal — e não em algum artefato espúrio de aquisição.

2.3.3 Importância de atributos em código

A Listagem 2.3 treina uma floresta aleatória sobre um problema de classificação de assinaturas e extrai a importância dos atributos — a contribuição relativa de cada característica do sinal para a decisão.

Listagem 2.3: Floresta aleatória e importância de atributos.

```
1 # Classificacao de assinaturas e leitura da importancia dos atributos.
```

```

2 import numpy as np
3 from sklearn.datasets import make_classification
4 from sklearn.model_selection import train_test_split
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.metrics import f1_score
7
8 X, y = make_classification(
9     n_samples=1500, n_features=12, n_informative=6, random_state=0
10 )
11 nomes = [f"banda_{j}" for j in range(X.shape[1])]
12
13 X_tr, X_te, y_tr, y_te = train_test_split(
14     X, y, test_size=0.25, random_state=0, stratify=y
15 )
16
17 floresta = RandomForestClassifier(
18     n_estimators=300, max_depth=None, random_state=0, n_jobs=-1
19 )
20 floresta.fit(X_tr, y_tr)
21 y_pred = floresta.predict(X_te)
22 print("F1:", round(f1_score(y_te, y_pred), 3))
23
24 # Importancia dos atributos, ordenada do mais ao menos influente
25 ordem = np.argsort(floresta.feature_importances_)[::-1]
26 print("\nImportancia dos atributos:")
27 for j in ordem[:6]:
28     print(f" {nomes[j]:>9}: {floresta.feature_importances_[j]:.3f}")

```

A floresta não exige padronização e tende a alcançar F1 elevado sem qualquer ajuste de hiperparâmetros — daí seu papel de *baseline* forte para classificação. A leitura da importância dos atributos, contudo, exige cautela: ela indica *associação* com a decisão, não *causalidade*, e pode ser inflada por atributos de alta cardinalidade. É um ponto de partida para a investigação, não uma conclusão.

2.4 Máquinas de vetores de suporte

A última família clássica que tratamos — as *máquinas de vetores de suporte* (*support vector machines*, SVM) — parte de uma intuição geométrica distinta e particularmente elegante, e mantém relevância em um nicho importante: problemas com *muitos atributos e poucos exemplos*, comuns quando a rotulagem é cara, como na classificação de assinaturas raras.

2.4.1 Margem máxima

Considere um problema de classificação binária linearmente separável. Há, em geral, infinitas retas (ou hiperplanos, em dimensão maior) que separam as duas classes. A SVM escolhe um critério específico: o hiperplano que *maximiza a margem* — a distância até os exemplos mais próximos de cada classe. Esses exemplos críticos, que tocam a margem, são os *vetores de suporte*, e dão nome ao método. A intuição é que o separador de margem máxima é o mais *robusto*: por estar o mais longe possível dos dados de ambas as classes, é o que menos provavelmente erra em exemplos novos próximos da fronteira.

Formalmente, com rótulos $y_i \in \{-1, +1\}$, busca-se

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{sujeito a} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n.$$

Minimizar $\|\mathbf{w}\|$ equivale a maximizar a margem. Na prática, como os dados raramente

são perfeitamente separáveis, admite-se uma *margem suave* (*soft margin*), introduzindo variáveis de folga que permitem algumas violações, penalizadas por um hiperparâmetro C que regula a troca entre margem larga e erros no treino.

2.4.2 O truque do *kernel*

A elegância da SVM revela-se quando os dados *não* são linearmente separáveis no espaço original. Em vez de construir explicitamente atributos não lineares, a SVM usa o *truque do kernel*: substitui o produto interno $\mathbf{x}_i^\top \mathbf{x}_j$ por uma *função de kernel* $K(\mathbf{x}_i, \mathbf{x}_j)$ que corresponde a um produto interno em um espaço de dimensão muito maior — sem nunca calcular as coordenadas nesse espaço. O *kernel* mais usado é o RBF (*radial basis function*, ou gaussiano):

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2).$$

Com ele, fronteiras de decisão arbitrariamente complexas tornam-se acessíveis, ao custo de dois hiperparâmetros a ajustar — C e γ — e de uma sensibilidade real a esse ajuste.

Listagem 2.4: SVM com *kernel* RBF e padronização acoplada.

```

1  # SVM com kernel RBF; padronizacao e' obrigatoria.
2  from sklearn.datasets import make_classification
3  from sklearn.model_selection import train_test_split
4  from sklearn.pipeline import make_pipeline
5  from sklearn.preprocessing import StandardScaler
6  from sklearn.svm import SVC
7  from sklearn.metrics import classification_report
8
9  X, y = make_classification(
10     n_samples=1200, n_features=20, n_informative=8, random_state=0
11 )
12 X_tr, X_te, y_tr, y_te = train_test_split(
13     X, y, test_size=0.25, random_state=0, stratify=y
14 )
15
16 # O pipeline garante que a padronizacao do teste use estatisticas do treino
17 modelo = make_pipeline(
18     StandardScaler(),
19     SVC(kernel="rbf", C=10.0, gamma="scale", class_weight="balanced")
20 )
21 modelo.fit(X_tr, y_tr)
22 print(classification_report(y_te, modelo.predict(X_te), digits=3))

```

Boa prática

Encapsular a padronização e o modelo em um único Pipeline do scikit-learn não é mero estilo: é a defesa mais eficaz contra a *fuga de informação* discutida no Capítulo 1. Sem o *pipeline*, é fácil padronizar o conjunto inteiro *antes* de separar treino e teste, deixando estatísticas do teste vazarem para o treino e produzindo métricas otimistas que não se sustentam em produção. Com o *pipeline*, a transformação é ajustada apenas no treino e reaplicada ao teste — automaticamente, a cada partição da validação cruzada.

2.4.3 Quando preferir cada família

Não há modelo universalmente melhor; há adequação ao problema. Como heurística inicial: a **regressão logística** é a primeira escolha quando se deseja interpretabilidade e

probabilidades calibradas, e quando a fronteira é aproximadamente linear; a **floresta aleatória** é a *baseline* robusta para dados tabulares heterogêneos de sensores, sem necessidade de padronização e com importância de atributos de brinde; a **SVM com kernel** brilha em alta dimensionalidade com poucos exemplos, mas exige padronização e ajuste cuidadoso de C e γ , e não escala bem para conjuntos muito grandes. A decisão final raramente se toma no papel: toma-se comparando os modelos empiricamente, como faremos a seguir.

2.5 Aplicação integrada: classificação de ameaças a partir de sensores

Encerramos o capítulo reunindo as quatro famílias em um único *pipeline* comparativo — o gesto metodológico central de todo projeto operacional. Simulamos um problema de *classificação de ameaças* a partir de dados de sensores: cada exemplo reúne características extraídas de um contato (parâmetros cinemáticos, atributos espectrais, índices de qualidade do sinal), e a tarefa é classificá-lo como ameaça ou não, sob desbalanceamento realista.

A Listagem 2.5 treina e avalia, sob *validação cruzada estratificada*, os quatro modelos do capítulo, ordenando-os pelo F1-score da classe de interesse — a métrica que melhor equilibra precisão e revocação na detecção da ameaça.

Listagem 2.5: Comparação sistemática de modelos clássicos sob validação cruzada.

```

1  # Comparacao de modelos classicos sob validacao cruzada estratificada.
2  import numpy as np
3  from sklearn.datasets import make_classification
4  from sklearn.model_selection import StratifiedKFold, cross_val_score
5  from sklearn.pipeline import make_pipeline
6  from sklearn.preprocessing import StandardScaler
7  from sklearn.linear_model import LogisticRegression
8  from sklearn.ensemble import RandomForestClassifier
9  from sklearn.svm import SVC
10
11 # Problema sintetico: deteccao de ameaca, 15% de exemplos positivos
12 X, y = make_classification(
13     n_samples=2500, n_features=16, n_informative=8,
14     weights=[0.85, 0.15], random_state=0
15 )
16
17 modelos = {
18     "Regressao logistica": make_pipeline(
19         StandardScaler(),
20         LogisticRegression(max_iter=1000, class_weight="balanced")),
21     "Floresta aleatoria": RandomForestClassifier(
22         n_estimators=300, class_weight="balanced",
23         random_state=0, n_jobs=-1),
24     "SVM (RBF)": make_pipeline(
25         StandardScaler(),
26         SVC(kernel="rbf", C=10.0, gamma="scale", class_weight="balanced")),
27 }
28
29 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
30 resultados = {}
31 for nome, modelo in modelos.items():
32     f1 = cross_val_score(modelo, X, y, cv=cv, scoring="f1")
33     resultados[nome] = (f1.mean(), f1.std())
34
35 print("Modelo          F1 medio    desvio")
36 for nome, (media, dp) in sorted(
37     resultados.items(), key=lambda kv: kv[1][0], reverse=True):

```

38

```
print(f" {nome:<22} {media:6.3f} +/- {dp:.3f}")
```

O resultado típico desse experimento confirma a lição central do capítulo: a floresta aleatória costuma liderar ou empatar em problemas tabulares heterogêneos, com pouco ou nenhum ajuste; a SVM com *kernel* se aproxima quando há estrutura não linear forte; e a regressão logística, embora muitas vezes inferior em F1, permanece valiosa por sua interpretabilidade e por suas probabilidades. A escolha final pesa não só o F1 médio, mas também sua *variância* (a estabilidade entre partições), o custo de inferência e — sempre, em defesa — a auditabilidade.

Contexto de defesa

Repare no que o *pipeline* comparativo *não* faz: ele não elege um vencedor de forma automática. Em um sistema crítico, a métrica é apenas uma das entradas da decisão. Um modelo 1% superior em F1, mas inaudível, que exige *hardware* dedicado de inferência e cujo comportamento se degrada sob *drift* de sensor, pode ser a pior escolha operacional, ainda que a melhor no *notebook*. A engenharia de defesa começa onde a competição de métricas termina.

2.6 O caminho à frente

Este capítulo cobriu o arsenal clássico do aprendizado supervisionado — regressão linear e logística, árvores e florestas, máquinas de vetores de suporte — e, mais importante que qualquer modelo isolado, o método de *compará-los honestamente* sob validação cruzada, com métricas adequadas ao custo operacional do erro.

No Capítulo 3, fecharemos o Módulo I com o salto para o *deep learning*: do perceptron ao *multilayer perceptron*, do algoritmo de retropropagação às primeiras arquiteturas profundas, com implementação em TensorFlow e PyTorch. Veremos que as redes neurais não tornam obsoletos os modelos deste capítulo — elas ampliam o repertório, e a disciplina de comparar contra uma *baseline* clássica forte permanece tão necessária quanto antes. O Módulo II, em seguida, levará essas redes ao terreno da percepção: visão computacional e linguagem natural aplicadas a problemas operacionais.

Resumo do capítulo

- A **regressão linear** modela uma saída contínua como combinação linear dos atributos. Sua transparência — coeficientes com interpretação direta — faz dela a *baseline* natural de todo problema de regressão, e instrumento de auditoria em manutenção preditiva.
- A **regressão logística** adapta a combinação linear à classificação, devolvendo *probabilidades* via função sigmoide. A probabilidade é informação operacional, e o *limiar* de decisão é uma alavanca a calibrar — não um valor fixo.
- Em **classes desbalanceadas** — a regra, e não a exceção, em detecção de ameaças —, a acurácia agregada engana. Avalie por precisão, revocação, F1 e matriz de confusão, ponderando o *custo operacional* distinto de falsos positivos e falsos negativos.

- **Árvores de decisão** capturam relações não lineares e são legíveis, mas sobreajustam. A **floresta aleatória** corrige isso por *bagging* e amostragem de atributos, oferecendo desempenho robusto e *importância de atributos* — valiosa para auditoria.
- As **máquinas de vetores de suporte** buscam o separador de *margem máxima* e, via *truque do kernel* (RBF), acessam fronteiras não lineares. Brilham em alta dimensão com poucos exemplos, mas exigem padronização e ajuste de C e γ .
- Não há modelo universalmente superior. A prática correta é **comparar modelos empiricamente**, sob validação cruzada estratificada, e decidir pesando F1, variância, custo de inferência e auditabilidade — pois, em defesa, a melhor escolha de *notebook* nem sempre é a melhor escolha operacional.

Armadilhas comuns

- **Avaliar problema desbalanceado por acurácia.** Com classe rara, responder sempre “classe majoritária” produz acurácia alta e utilidade nula. Use métricas por classe e analise a matriz de confusão à luz do custo operacional de cada tipo de erro.
- **Esquecer de padronizar os atributos.** Regressão logística e SVM são sensíveis à escala. Não padronizar prejudica — às vezes gravemente — a otimização. Encapsule a padronização no *pipeline* para que seja sempre aplicada de forma correta.
- **Padronizar antes de separar treino e teste.** Ajustar o padronizador sobre o conjunto inteiro deixa estatísticas do teste vazarem para o treino (*data leakage*). Ajuste sempre no treino e reaplique ao teste — o que o Pipeline faz automaticamente.
- **Confiar cegamente na importância de atributos.** A importância da floresta indica associação, não causalidade, e pode ser inflada por atributos de alta cardinalidade. É ponto de partida de investigação, não veredito.
- **Pular a *baseline*.** Apresentar um modelo complexo sem compará-lo a uma referência simples impede saber se ele de fato ajuda — e esconde erros de *pipeline* que uma *baseline* revelaria de imediato.
- **Tratar o limiar de 0,5 como sagrado.** O limiar de decisão da regressão logística é ajustável e deve refletir o custo relativo dos erros. Em ameaças raras de alto custo, frequentemente se baixa o limiar para aumentar a revocação, aceitando mais falsos positivos.

Exercícios

Essencial — fixação, com solução guiada no repositório

Exercício 2.1 Execute a Listagem 2.1. Compare os coeficientes estimados com os valores que geraram os dados (0,18; 7,5; 0,9). Em seguida, aumente o ruído (o desvio da `rng.normal`) de 12 para 40 e relate, em duas linhas, o efeito sobre o R^2 e sobre a proximidade dos coeficientes.

Exercício 2.2 Na Listagem 2.2, remova o parâmetro `class_weight="balanced"` e reexecute. Compare a revocação da classe *hostil* antes e depois. Explique, em uma frase, por

que o modelo sem ponderação tende a “ignorar” a classe rara.

Exercício 2.3 Execute a Listagem 2.3 e liste os três atributos mais importantes. Em seguida, treine uma *única* árvore de decisão (`DecisionTreeClassifier`) sobre os mesmos dados e compare o F1 ao da floresta. Em duas linhas, relacione a diferença observada ao conceito de *bagging*.

Tático — aplicação a um problema semelhante

Exercício 2.4 Na Listagem 2.4, faça uma busca manual variando o hiperparâmetro *C* entre {0,1; 1; 10; 100} e registre o F1 da classe positiva em cada caso. Descreva, em poucas linhas, o comportamento observado e relacione-o à troca entre margem larga e erros no treino regulada por *C*.

Exercício 2.5 Estenda a Listagem 2.5 acrescentando, à comparação, a curva *precision–recall* do melhor modelo (use `precision_recall_curve` sobre as probabilidades previstas). Identifique um limiar que privilegie a revocação da ameaça acima de 0,90 e relate o preço pago em precisão.

Exercício 2.6 Considere o problema operacional de *previsão de falha de componente* a partir de histórico de operação (variáveis contínuas de temperatura, vibração e horas de uso). Decida se você o atacaria como regressão (tempo até a falha) ou classificação (falha nas próximas *N* horas), justifique a escolha e indique, para cada caso, a métrica de avaliação adequada e o motivo.

Estratégico — extensão criativa

Exercício 2.7 Em um breve texto técnico (até duas páginas), discuta a tensão entre *desempenho* e *auditabilidade* na escolha de um modelo para um sistema de apoio à triagem de contatos. Construa um argumento, fundamentado nos conceitos do capítulo, sobre em que circunstâncias um modelo menos preciso, mas interpretável, deve ser preferido a um modelo mais preciso e opaco — e proponha critérios objetivos para essa decisão.

Exercício 2.8 Projete (em pseudocódigo ou diagrama) um *pipeline* de validação que respeite a estrutura temporal de dados de sensores — isto é, que evite usar informação futura para prever o passado. Explique por que a validação cruzada estratificada *padrão* da Listagem 2.5 seria inadequada para uma série temporal, e que estratégia (por exemplo, validação por janelas deslizantes) a substituiria.

Exercício 2.9 Escolha um conjunto de dados tabular de domínio público (de sensores, logística ou meteorologia) e conduza um estudo comparativo completo dos quatro modelos do capítulo, redigindo um miniartigo de até três páginas que documente: a definição do problema, a estratégia de avaliação, os resultados com sua variância, e uma recomendação fundamentada — incluindo a discussão explícita dos vieses do conjunto e das limitações de generalização para um contexto operacional real.

Redes Neurais Artificiais e Deep Learning

Objetivos do capítulo

Ao final deste capítulo, você será capaz de:

- explicar o *perceptron*, o seu limite fundamental (o problema do XOR) e como o *perceptron multicamadas* (MLP) o supera pela composição de camadas com ativação não linear;
- descrever o *gradiente descendente* e o algoritmo de *retropropagação* como o mecanismo pelo qual uma rede aprende, e compreender o papel das funções de ativação e o problema do gradiente evanescente;
- distinguir as principais arquiteturas profundas — *redes convolucionais* (CNN), *recorrentes* (RNN) e *LSTM* — e o tipo de dado a que cada uma se adéqua;
- implementar e treinar redes em TensorFlow/Keras e em PyTorch, reconhecendo os idiomas declarativo e imperativo de cada biblioteca;
- aplicar *regularização* (*dropout*, *weight decay*, *batch normalization*), otimizadores modernos e boas práticas de treinamento para controlar o sobreajuste em problemas operacionais.

O Capítulo 2 percorreu o arsenal clássico do aprendizado supervisionado e, mais importante que qualquer modelo isolado, firmou uma disciplina: comparar modelos honestamente contra uma *baseline*, sob validação cruzada, com métricas ajustadas ao custo operacional do erro. Neste capítulo damos o salto para as **redes neurais profundas** — a família de modelos que, na última década, redefiniu o que é possível em percepção (visão e linguagem) e que constitui o motor dos Módulos II e III. Com ele, fechamos o Módulo I.

Uma advertência abre o capítulo, e é a mesma que fechou o anterior: profundidade não é sofisticação gratuita. Uma rede profunda só se justifica, em um sistema crítico, quando supera com folga mensurável a *baseline* clássica — e o faz ao custo de menor auditabilidade, tema que o Módulo IV nos obrigará a enfrentar de frente. O aumento de

capacidade dos modelos não dissolve a disciplina do Capítulo 2; torna-a mais necessária.

3.1 Do perceptron ao perceptron multicamadas

A tradição conexionista, apresentada no Capítulo 1, nasce de uma metáfora biológica: se o cérebro computa por meio de neurônios que se excitam e se inibem, talvez se possa aprender construindo unidades de cálculo análogas. O modelo de McCulloch e Pitts (1943) e o *perceptron* de Rosenblatt (1958) são as sementes dessa ideia — e o ponto de partida natural para entender o *deep learning*.

3.1.1 O perceptron

O perceptron é a unidade elementar. Dado um vetor de entrada $\mathbf{x} = (x_1, \dots, x_d)$, um vetor de pesos $\mathbf{w}$ e um viés b , ele calcula uma combinação linear e a submete a uma função de ativação em degrau:

$$\hat{y} = \varphi(\mathbf{w}^\top \mathbf{x} + b), \quad \varphi(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

Geometricamente, $\mathbf{w}^\top \mathbf{x} + b = 0$ é um hiperplano que divide o espaço de entrada em dois semiespaços; o perceptron classifica conforme o lado. O aprendizado ajusta os pesos por uma regra simples e local: a cada exemplo mal classificado,

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y - \hat{y}) \mathbf{x},$$

onde η é a *taxa de aprendizado*. O *teorema de convergência do perceptron* garante que, se os dados forem *linearmente separáveis*, esse processo termina em um número finito de passos. O leitor reconhecerá aqui o ancestral da regressão logística do Capítulo 2 — a diferença é que a logística substitui o degrau por uma sigmoide suave e devolve probabilidades, o que a torna treinável por gradiente e muito mais informativa.

3.1.2 O limite do perceptron: o problema do XOR

A limitação é séria e histórica. Minsky e Papert, em *Perceptrons* (1969), demonstraram que um único perceptron não representa a função lógica **XOR** (ou-exclusivo): não existe reta que separe os pares $\{(0,0), (1,1)\}$ de $\{(0,1), (1,0)\}$, porque o problema não é linearmente separável. A repercussão desse resultado é frequentemente apontada como um dos gatilhos do primeiro “inverno da IA” discutido no Capítulo 1: o financiamento à abordagem conexionista minguou por mais de uma década.

A saída — conhecida em princípio, mas só viável na prática com a retropropagação — é *empilhar* unidades em camadas, intercalando uma **não linearidade**. Uma camada intermediária constrói representações novas do dado, em que o problema, antes emaranhado, torna-se separável. É esse o salto conceitual que funda o aprendizado profundo.

3.1.3 O perceptron multicamadas (MLP)

O *perceptron multicamadas* organiza neurônios em camadas sucessivas. Com uma camada oculta, a rede calcula

$$\mathbf{h} = \varphi(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \hat{\mathbf{y}} = \psi(\mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}),$$

em que φ é uma ativação não linear (tipicamente a ReLU, adiante) e ψ é a ativação de saída — sigmoide para decisão binária, *softmax* para classificação multiclasse. As camadas

intermediárias não são observadas diretamente: são *representações latentes* que a rede aprende para tornar a tarefa solúvel.

O poder dessa construção é formalizado pelo **teorema da aproximação universal** (Cybenko, 1989; Hornik, 1991): uma rede com uma única camada oculta e número suficiente de unidades aproxima qualquer função contínua sobre um conjunto compacto, com a precisão que se queira. O teorema é de existência, não de eficiência — na prática, *profundidade* (muitas camadas) costuma ser mais econômica em parâmetros do que *largura* (uma camada enorme), e é a profundidade que dá nome ao *deep learning*.

Contexto de defesa

Em defesa, o MLP é a ferramenta natural para a *fusão de dados tabulares* de múltiplos sensores — combinar leituras heterogêneas (radar, acústica, sinais) em uma decisão única. Para dados com estrutura espacial (imagens) ou temporal (séries, trajetórias), porém, arquiteturas especializadas, tratadas na Seção 3.3, exploram essa estrutura e superam largamente um MLP genérico — tanto em desempenho quanto em economia de parâmetros, o que importa diretamente para a IA embarcada do Módulo IV.

3.2 Retropropagação e o treinamento por gradiente

Uma rede só é útil depois de treinada. Treinar significa ajustar todos os pesos $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}$ de modo a minimizar uma função de perda que meça o erro das previsões. O mecanismo que torna isso possível em redes profundas é a *retropropagação*.

3.2.1 A função de perda e o gradiente descendente

Seja $\mathcal{L}(\theta)$ a perda média sobre os dados de treino — entropia cruzada para classificação, erro quadrático para regressão (Capítulo 2). O **gradiente descendente** atualiza os parâmetros na direção que mais reduz a perda:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta),$$

onde $\nabla_{\theta} \mathcal{L}$ é o vetor de derivadas parciais da perda em relação a cada parâmetro e η é a taxa de aprendizado. Calcular o gradiente sobre *todo* o conjunto a cada passo é caro; na prática usa-se o **gradiente descendente estocástico em minilotes** (*mini-batch SGD*), que estima o gradiente sobre pequenos subconjuntos. Além de barato, o ruído dessa estimativa ajuda a escapar de mínimos rasos.

3.2.2 O algoritmo de retropropagação

O obstáculo é calcular $\nabla_{\theta} \mathcal{L}$ para uma rede com muitas camadas. A **retropropagação** resolve isso aplicando a *regra da cadeia* de forma organizada, propagando o erro da saída para as entradas. O procedimento tem dois passos. No *passo para frente*, calculam-se as ativações de cada camada até a saída e a perda. No *passo para trás*, calcula-se um sinal de erro δ na saída e propaga-se para trás:

$$\delta^{(l)} = \left(\mathbf{W}^{(l+1)} \right)^{\top} \delta^{(l+1)} \odot \varphi' \left(\mathbf{z}^{(l)} \right), \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} \left(\mathbf{a}^{(l-1)} \right)^{\top},$$

onde $\mathbf{z}^{(l)}$ é a pré-ativação da camada l , $\mathbf{a}^{(l-1)}$ é a saída da camada anterior e $\odot$ é o produto elemento a elemento. Não é preciso decorar essas fórmulas: as bibliotecas as executam

automaticamente por *diferenciação automática*. Mas entendê-las explica os modos de falha — em particular, por que gradientes podem “desaparecer” ao atravessar muitas camadas.

3.2.3 Funções de ativação

A não linearidade φ é indispensável: empilhar camadas puramente lineares colapsa em um único mapa linear, sem ganho de expressividade. As escolhas clássicas são três.

- **Sigmoide**, $\sigma(z) = 1/(1 + e^{-z})$: comprime a $[0, 1]$, mas *satura* — sua derivada não passa de 0,25 e tende a zero nos extremos. Em redes profundas, isso provoca o **gradiente evanescente**: o sinal de erro encolhe a cada camada e as primeiras camadas praticamente não aprendem.
- **Tangente hiperbólica**, $\tanh(z)$: semelhante à sigmoide, porém centrada em zero, o que ajuda a otimização; ainda satura.
- **ReLU**, $\varphi(z) = \max(0, z)$: derivada igual a 1 para $z > 0$, o que *preserva* o gradiente e acelera o treino; é barata e induz ativações esparsas. Tornou-se o padrão nas camadas ocultas. Seu risco — a “ReLU morta”, unidade travada em zero — é mitigado por variantes como a *Leaky ReLU*.

Na *saída*, a ativação segue a tarefa: sigmoide para decisão binária, *softmax* para multiclasse. A Listagem 3.1 treina um MLP sobre o mesmo problema de triagem de ameaças do Capítulo 2 — o que permite, nos exercícios, confrontá-lo diretamente com a *baseline* logística.

Listagem 3.1: Perceptron multicamadas em Keras para triagem de ameaças.

```

1 import numpy as np
2 import tensorflow as tf
3 from tensorflow import keras
4 from tensorflow.keras import layers
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7
8 # Dados sintéticos de triagem: n amostras, d atributos de sensor.
9 # A classe positiva (ameaça) é rara e depende de forma NÃO LINEAR
10 # dos atributos -- terreno onde uma rede pode superar a baseline.
11 rng = np.random.default_rng(42)
12 n, d = 4000, 12
13 X = rng.normal(size=(n, d))
14 logit = 1.8 * X[:, 0] * X[:, 3] - 1.2 * X[:, 5] + 0.9 * np.sin(X[:, 7])
15 prob = 1.0 / (1.0 + np.exp(-logit))
16 y = (rng.uniform(size=n) < prob * 0.35).astype("int32")
17
18 X_tr, X_te, y_tr, y_te = train_test_split(
19     X, y, test_size=0.25, stratify=y, random_state=42)
20
21 # Padronização ajustada SO no treino (evita vazamento -- ver Cap. 2).
22 scaler = StandardScaler().fit(X_tr)
23 X_tr, X_te = scaler.transform(X_tr), scaler.transform(X_te)
24
25 # MLP: duas camadas ocultas com ReLU; saída sigmoide (probabilidade).
26 model = keras.Sequential([
27     layers.Input(shape=(d,)),
28     layers.Dense(64, activation="relu"),
29     layers.Dense(32, activation="relu"),
30     layers.Dense(1, activation="sigmoid"),
31 ])

```

```

32
33 model.compile(
34     optimizer=keras.optimizers.Adam(learning_rate=1e-3),
35     loss="binary_crossentropy",
36     metrics=[keras.metrics.Recall(name="revocacao"),
37              keras.metrics.Precision(name="precisao")])
38
39 # Pesos de classe compensam o desbalanceamento (análogo ao Cap. 2).
40 n_pos = max(int((y_tr == 1).sum()), 1)
41 peso = {0: 1.0, 1: float((y_tr == 0).sum() / n_pos)}
42
43 model.fit(X_tr, y_tr, validation_split=0.2, epochs=30,
44         batch_size=64, class_weight=peso, verbose=0)
45
46 _, rev, prec = model.evaluate(X_te, y_te, verbose=0)
47 print(f"Revocacao (ameaca): {rev:.3f}   Precisao: {prec:.3f}")

```

Nota

As duas grandes bibliotecas de *deep learning* adotam idiomas distintos. TensorFlow/Keras é **declarativo**: descreve-se a arquitetura em alto nível e o método `fit` cuida do laço de treino — rápido para prototipar e implantar. PyTorch é **imperativo** (*define-by-run*): o grafo é construído a cada execução, em Python puro, o que dá controle fino e favorece a pesquisa. Ambos são padrão de mercado; a proposta desta obra emprega os dois. A Listagem 3.2 reescreve a mesma rede em PyTorch, tornando a retropropagação *visível* no código.

Listagem 3.2: A mesma rede em PyTorch: o laço de treino explícito.

```

1  import torch
2  import torch.nn as nn
3
4  # Reaproveita X_tr, y_tr já padronizados, convertidos em tensores.
5  Xt = torch.tensor(X_tr, dtype=torch.float32)
6  yt = torch.tensor(y_tr, dtype=torch.float32).unsqueeze(1)
7
8  # Mesmo MLP no idioma imperativo do PyTorch.
9  rede = nn.Sequential(
10     nn.Linear(d, 64), nn.ReLU(),
11     nn.Linear(64, 32), nn.ReLU(),
12     nn.Linear(32, 1),      # sem sigmoide: usamos BCEWithLogits
13 )
14
15 criterio = nn.BCEWithLogitsLoss()
16 otimizador = torch.optim.Adam(rede.parameters(), lr=1e-3)
17
18 # Laço de treino explícito: a retropropagacao aparece em tres linhas.
19 for epoca in range(30):
20     otimizador.zero_grad()      # 1) zera os gradientes acumulados
21     logits = rede(Xt)           # 2) passo para frente
22     perda = criterio(logits, yt) # calcula a perda
23     perda.backward()            # 3) retropropagacao (autograd)
24     otimizador.step()           # atualiza os pesos
25
26 print(f"Perda final de treino: {perda.item():.4f}")

```

3.3 Arquiteturas profundas

Um MLP trata a entrada como um vetor sem estrutura: se as colunas forem embaralhadas de forma consistente, ele aprende igualmente bem. Mas imagens têm estrutura *espacial* — pixels vizinhos se relacionam — e séries têm estrutura *temporal*. Arquiteturas especializadas embutem essas suposições no próprio desenho da rede, ganhando desempenho e economizando parâmetros.

3.3.1 Redes neurais convolucionais (CNN)

A **rede convolucional** é o padrão para imagens. Seu bloco central é a *convolução*: um pequeno filtro (*kernel*) desliza sobre a imagem e calcula, em cada posição, uma soma ponderada local, produzindo um *mapa de ativação*. Para uma imagem I e um filtro K ,

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n).$$

Três ideias explicam o seu sucesso. Os *campos receptivos locais*: cada unidade enxerga apenas uma vizinhança, adequada à natureza local das feições de imagem. O *compartilhamento de pesos*: o mesmo filtro é aplicado em toda a imagem, o que confere *invariância à translação* (um objeto é reconhecido onde quer que apareça) e reduz drasticamente o número de parâmetros frente a uma camada densa. E o *pooling* (agregação, tipicamente pelo máximo), que reduz a resolução e adiciona robustez a pequenos deslocamentos. Empilhando blocos de convolução, ReLU e *pooling*, a rede constrói uma hierarquia de feições: bordas, texturas, partes e, por fim, objetos.

Contexto de defesa

As CNNs são a espinha dorsal da detecção de alvos em imagens — de satélite, de radar de abertura sintética (SAR) e de vídeo em movimento captado por VANT. O *Project Maven*, do Departamento de Defesa dos EUA, foi precisamente a aplicação de visão computacional profunda a vídeo de sensor, com o objetivo declarado de reduzir a sobrecarga dos analistas na triagem de imagens. O Capítulo 4 desenvolve esse terreno com arquiteturas de detecção como YOLO e Faster R-CNN, que estendem os princípios convolucionais vistos aqui.

Listagem 3.3: CNN compacta em Keras para classificação de imagens.

```

1 from tensorflow import keras
2 from tensorflow.keras import layers
3
4 # CNN para imagens em tons de cinza de 64x64x1: alvo presente ou ausente.
5 H, W = 64, 64
6 cnn = keras.Sequential([
7     layers.Input(shape=(H, W, 1)),
8
9     layers.Conv2D(16, 3, padding="same", activation="relu"),
10    layers.MaxPooling2D(),           # 64x64 -> 32x32
11
12    layers.Conv2D(32, 3, padding="same", activation="relu"),
13    layers.MaxPooling2D(),           # 32x32 -> 16x16
14
15    layers.Conv2D(64, 3, padding="same", activation="relu"),
16    layers.MaxPooling2D(),           # 16x16 -> 8x8
17
```

```

18     layers.GlobalAveragePooling2D(),
19     layers.Dense(64, activation="relu"),
20     layers.Dense(1, activation="sigmoid"),
21 ])
22
23 cnn.compile(optimizer="adam", loss="binary_crossentropy",
24             metrics=["accuracy"])
25 # Note, no sumário, a economia de parametros frente a uma rede densa
26 # equivalente sobre a imagem achatada (64*64 = 4096 entradas).
27 cnn.summary()

```

3.3.2 Redes recorrentes e LSTM

Para *sequências*, a arquitetura natural é a **rede recorrente** (RNN). Ela processa a entrada passo a passo, mantendo um *estado oculto* $\mathbf{h}_t$ que funciona como memória do que já foi visto:

$$\mathbf{h}_t = \varphi(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}).$$

O problema é que, ao retropropagar por sequências longas, o gradiente tende a desaparecer ou a explodir — a RNN simples “esquece” dependências distantes. A **LSTM** (*Long Short-Term Memory*) corrige isso com um *estado de célula* $\mathbf{c}_t$ e três *comportas* que regulam o fluxo de informação — de esquecimento ($\mathbf{f}_t$), de entrada ($\mathbf{i}_t$) e de saída ($\mathbf{o}_t$):

$$\begin{aligned}
 \mathbf{f}_t &= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f), \\
 \mathbf{i}_t &= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \\
 \mathbf{o}_t &= \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \\
 \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c), \\
 \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t).
 \end{aligned}$$

A comporta de esquecimento decide o que descartar da memória; a de entrada, o que acrescentar; a de saída, o que expor. Essa estrutura preserva o gradiente ao longo de horizontes longos. Em defesa, RNNs e LSTMs modelam *séries temporais* de telemetria, previsão de *trajetórias* (faixas de radar, dados AIS) e, historicamente, texto — tema do Capítulo 5.

Boa prática

Para muitas tarefas de sequência, sobretudo em texto, a arquitetura *Transformer* (Capítulo 6) hoje supera as redes recorrentes e paraleliza melhor o treino. A LSTM, porém, permanece forte e *barata* em sequências curtas e em plataformas com restrição de recursos — exatamente o cenário da IA embarcada do Módulo IV. A escolha da arquitetura é, também aqui, uma decisão de engenharia condicionada pelo custo operacional, não apenas pelo estado da arte.

3.4 Regularização, otimização e boas práticas de treinamento

Redes profundas têm capacidade enorme — e, com ela, a tendência a *decorar* os dados de treino em vez de aprender a regularidade que generaliza. Controlar essa tendência é o que separa um modelo de *notebook* de um modelo operacional.

3.4.1 Sobreajuste em redes profundas

O sintoma é inequívoco: a perda de *treino* continua a cair enquanto a perda de *validação* estaciona e depois sobe. A rede está memorizando ruído. Por isso, a primeira regra é **monitorar sempre um conjunto de validação** separado — sem ele, o sobreajuste passa despercebido até o fracasso em produção.

3.4.2 Técnicas de regularização

Um repertório consolidado combate o sobreajuste.

- **Dropout**: durante o treino, desliga aleatoriamente uma fração p das unidades a cada passo, impedindo que se co-adaptem; na inferência, todas voltam, com a devida escala.
- **Weight decay** (regularização L_2): acrescenta $\lambda \|\theta\|^2$ à perda, favorecendo pesos menores e funções mais suaves.
- **Batch normalization**: normaliza as pré-ativações de cada camada por minilote, estabilizando e acelerando o treino, com efeito regularizador brando.
- **Parada antecipada** (*early stopping*): interrompe o treino quando a perda de validação deixa de melhorar e retém os melhores pesos.
- **Aumento de dados** (*data augmentation*): para imagens, espelhamentos, rotações e recortes aleatórios multiplicam efetivamente o conjunto e ensinam invariâncias — recurso precioso em defesa, onde imagens rotuladas são escassas.

3.4.3 Otimização

O SGD puro funciona, mas é lento e sensível. O *momento* acumula uma velocidade que suaviza a trajetória; o **Adam** adapta a taxa de aprendizado por parâmetro a partir de estimativas dos momentos do gradiente e é o otimizador-padrão de fato. Ainda assim, o hiperparâmetro mais importante é a **taxa de aprendizado**: alta demais, o treino diverge; baixa demais, ele se arrasta. *Agendas* de decaimento e aquecimento ajudam. O tamanho do minilote troca ruído por velocidade.

3.4.4 Boas práticas

Feçam o quadro as práticas que, em sistemas críticos, tocam a própria auditabilidade: *normalizar as entradas* (como no Capítulo 2); usar *inicialização* de pesos sensata (He para ReLU, Glorot para *tanh*); partir sempre de uma *baseline* forte; acompanhar as curvas de validação; e *fixar sementes* aleatórias para reprodutibilidade — em defesa, um resultado que não se reproduz não se audita. Por fim, manter o modelo tão pequeno quanto o problema permitir, antecipando as restrições de energia, memória e latência do Módulo IV.

Listagem 3.4: Treino disciplinado: regularização e retornos de validação.

```

1 from tensorflow import keras
2 from tensorflow.keras import layers
3
4 # CNN com regularizacao explicita: batch norm + dropout + weight decay.
5 modelo = keras.Sequential([
6     layers.Input(shape=(64, 64, 1)),
7
8     layers.Conv2D(32, 3, padding="same", use_bias=False),
9     layers.BatchNormalization(),           # estabiliza e acelera o treino
10    layers.Activation("relu"),

```



```

11     layers.MaxPooling2D(),
12
13     layers.Conv2D(64, 3, padding="same", use_bias=False),
14     layers.BatchNormalization(),
15     layers.Activation("relu"),
16     layers.MaxPooling2D(),
17
18     layers.GlobalAveragePooling2D(),
19     layers.Dropout(0.4),           # combate a co-adaptacao
20     layers.Dense(64, activation="relu",
21                 kernel_regularizer=keras.regularizers.l2(1e-4)),
22     layers.Dropout(0.3),
23     layers.Dense(1, activation="sigmoid"),
24 ])
25
26 modelo.compile(optimizer=keras.optimizers.Adam(1e-3),
27               loss="binary_crossentropy", metrics=["accuracy"])
28
29 # Parada antecipada pela perda de validacao + guarda do melhor modelo.
30 retornos = [
31     keras.callbacks.EarlyStopping(monitor="val_loss", patience=6,
32                                   restore_best_weights=True),
33     keras.callbacks.ModelCheckpoint("melhor_modelo.keras",
34                                     monitor="val_loss", save_best_only=True),
35 ]
36
37 # historico = modelo.fit(X_tr, y_tr, validation_split=0.2,
38 #                       epochs=100, batch_size=32, callbacks=retornos)

```

Armadilha comum

Recorrer ao *deep learning* por reflexo. Em dados tabulares de volume modesto — a situação frequente em defesa — uma rede profunda costuma *perder* para uma *baseline* clássica bem ajustada (Capítulo 2), além de custar mais para treinar, servir e, sobretudo, auditar. A pergunta correta não mudou: qual é o modelo mais simples que resolve o problema com garantias suficientes? A rede profunda deve ser a resposta a uma necessidade demonstrada, não o ponto de partida.

3.5 Aplicação integrada: uma CNN para triagem de imagens de sensor

Reunimos as peças em um problema realista de apoio à decisão. Um fluxo de vídeo de sensor gera mais quadros do que qualquer equipe de analistas consegue inspecionar. A tarefa não é substituir o analista, mas *priorizar* a sua atenção: sinalizar os quadros com maior probabilidade de conter um alvo de interesse — o mesmo objetivo declarado do *Project Maven*. A métrica operacional decisiva é a **revocação** da classe-alvo: é tolerável revisar alguns falsos positivos; é grave deixar passar um alvo real.

A Listagem 3.5 treina uma CNN de triagem com aumento de dados e parada antecipada e — passo inegociável desta obra — confronta-a com uma **baseline** clássica. Só a comparação justifica (ou não) o custo adicional da rede profunda em produção.

Listagem 3.5: Pipeline de triagem: CNN com aumento de dados versus baseline clássica.

```

1 import numpy as np
2 from tensorflow import keras
3 from tensorflow.keras import layers

```

```

4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.metrics import recall_score, precision_score
6
7 # Pressupoe imagens ja carregadas e normalizadas para [0, 1]:
8 # X_tr, X_te : arrays (n, 64, 64, 1) float32
9 # y_tr, y_te : rotulos em {0, 1}, com separacao estratificada previa.
10
11 # --- 1. Aumento de dados: invariâncias uteis, aplicado so no treino ---
12 aumento = keras.Sequential([
13     layers.RandomFlip("horizontal"),
14     layers.RandomRotation(0.05),
15     layers.RandomZoom(0.05),
16 ])
17
18 # --- 2. CNN de triagem (API funcional) ---
19 entrada = keras.Input(shape=(64, 64, 1))
20 x = aumento(entrada)
21 x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
22 x = layers.MaxPooling2D()(x)
23 x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
24 x = layers.MaxPooling2D()(x)
25 x = layers.GlobalAveragePooling2D()(x)
26 x = layers.Dropout(0.4)(x)
27 saida = layers.Dense(1, activation="sigmoid")(x)
28 cnn = keras.Model(entrada, saida)
29
30 # Otimiza para REVOCACAO: em triagem, o custo de perder um alvo domina.
31 cnn.compile(optimizer="adam", loss="binary_crossentropy",
32             metrics=[keras.metrics.Recall(name="rev")])
33 parada = keras.callbacks.EarlyStopping(monitor="val_rev", mode="max",
34                                       patience=8, restore_best_weights=True)
35 cnn.fit(X_tr, y_tr, validation_split=0.2, epochs=60,
36        batch_size=32, callbacks=[parada], verbose=0)
37 p_cnn = (cnn.predict(X_te, verbose=0).ravel() >= 0.5).astype(int)
38
39 # --- 3. BASELINE obrigatoria: RandomForest sobre pixels subamostrados ---
40 Xf_tr = X_tr.reshape(len(X_tr), -1)[: , ::16]
41 Xf_te = X_te.reshape(len(X_te), -1)[: , ::16]
42 base = RandomForestClassifier(n_estimators=300, class_weight="balanced",
43                             random_state=42).fit(Xf_tr, y_tr)
44 p_base = base.predict(Xf_te)
45
46 # --- 4. Comparacao sob a metrica operacional ---
47 for nome, pred in [("CNN", p_cnn), ("RandomForest", p_base)]:
48     r = recall_score(y_te, pred)
49     p = precision_score(y_te, pred, zero_division=0)
50     print(f"{nome:13s} revocacao={r:.3f} precisao={p:.3f}")

```

O desfecho operacional não termina no número. A CNN *tria*; o ser humano *decide*. Esse arranjo — o modelo prioriza, o operador julga — é a forma prática do princípio do **controle humano significativo**, que o Capítulo 10 tratará como exigência normativa. Uma triagem que aumenta a revocação sem substituir o julgamento humano amplia a capacidade da equipe; uma automação que o dispensa transfere ao modelo uma responsabilidade que ele não pode carregar.

3.6 O caminho à frente

Com o perceptron, a retropropagação e as arquiteturas convolucional e recorrente, temos o motor do aprendizado profundo — e, com ele, fechamos o Módulo I. O Módulo II põe esse motor a serviço da *percepção*. O Capítulo 4 desenvolve a visão computacional para defesa, levando adiante as CNNs vistas aqui até arquiteturas de detecção de alvos (YOLO, Faster R-CNN) e o tratamento de imagens SAR e de VANT. O Capítulo 5 trata do processamento de linguagem natural para inteligência. E o Capítulo 6 chega aos modelos de linguagem de grande escala e à arquitetura *Transformer*, que hoje supera as redes recorrentes em quase toda tarefa de sequência.

A disciplina que atravessou os três capítulos do Módulo I — comparar contra uma *baseline* honesta, escolher métricas ao custo operacional do erro, manter o humano no centro da decisão — não se dissolve à medida que os modelos ganham capacidade. Torna-se, a cada capítulo, mais indispensável.

Resumo do capítulo

- O **perceptron** classifica por um hiperplano e só resolve problemas linearmente separáveis — o **XOR** expõe seu limite. O **perceptron multicamadas**, ao intercalar não linearidades entre camadas, aproxima qualquer função contínua (aproximação universal).
- O treino minimiza a perda por **gradiente descendente**; a **retropropagação** calcula o gradiente pela regra da cadeia, propagando o erro para trás. A não linearidade é essencial, e a **ReLU** substituiu a sigmoide nas camadas ocultas por evitar o *gradiente evanescente*.
- As **redes convolucionais** exploram a estrutura espacial das imagens por campos receptivos locais, compartilhamento de pesos e *pooling*, construindo feições hierárquicas — são a base da visão computacional para defesa.
- As **redes recorrentes** e, sobretudo, as **LSTM** tratam sequências mantendo memória; as comportas da LSTM preservam o gradiente em horizontes longos, viabilizando séries temporais e trajetórias.
- Contra o sobreajuste, combinam-se **dropout**, **weight decay**, **batch normalization**, **parada antecipada** e **aumento de dados**; o **Adam** é o otimizador-padrão, e a **taxa de aprendizado**, o hiperparâmetro mais crítico.
- O *deep learning* **amplia** o repertório, mas não **substitui** a *baseline* clássica: uma rede profunda só se justifica quando a supera com folga, ao custo de menor auditabilidade — e a decisão final permanece humana.

Armadilhas comuns

- **Adotar *deep learning* por reflexo.** Em dados tabulares de volume modesto, uma *baseline* clássica bem ajustada costuma vencer a rede profunda, a custo menor e com maior auditabilidade. Exija que a rede *prove* sua vantagem.
- **Não normalizar as entradas.** Redes treinam mal com atributos em escalas díspares.

Padronize (e faça-o ajustando as estatísticas apenas no treino, para não vaziar informação do teste).

- **Empilhar camadas sem ativação não linear.** Uma pilha de camadas puramente lineares colapsa em um único mapa linear — toda a expressividade extra se perde. A não linearidade entre camadas é o que dá poder à rede.
- **Usar sigmoide nas camadas ocultas de uma rede profunda.** A saturação provoca gradiente evanescente e trava o aprendizado das primeiras camadas. Prefira ReLU (ou variantes) nas ocultas; reserve a sigmoide para a saída binária.
- **Treinar sem conjunto de validação.** Sem acompanhar a perda de validação, o sobreajuste avança em silêncio e só se revela — tarde — em produção. Monitore-a e use parada antecipada.
- **Tratar a rede como caixa-preta em sistema crítico.** Sem sementes fixas, versionamento e rastreabilidade, o resultado não se reproduz nem se audita — e, sem controle humano na decisão, a automação assume um risco que não lhe cabe (Capítulo 10).

Exercícios

Essencial — fixação, com solução guiada no repositório

Exercício 3.1 Execute a Listagem 3.1 e compare a revocação e a precisão do MLP às de uma regressão logística (Capítulo 2) sobre os *mesmos* dados. Em seguida, varie o número de unidades das camadas ocultas em $\{16, 64, 256\}$ e relate, em poucas linhas, o efeito sobre as duas métricas e sobre os sinais de sobreajuste.

Exercício 3.2 Na Listagem 3.2, remova as ativações `nn.ReLU()`, tornando a rede puramente linear. Reexecute e explique, em uma frase, por que o desempenho colapsa ao de um modelo linear — relacionando o resultado ao papel da não linearidade entre camadas.

Exercício 3.3 Execute `cnn.summary()` na Listagem 3.3 e anote o total de parâmetros. Calcule quantos parâmetros teria a *primeira* camada de uma rede densa que recebesse a imagem achatada ($64 \times 64 = 4096$ entradas) e projetasse em 16 unidades. Compare os números e explique, em duas linhas, a economia trazida pelo compartilhamento de pesos.

Tático — aplicação a um problema semelhante

Exercício 3.4 Usando a Listagem 3.4, treine o modelo em um conjunto de imagens de sua escolha *com* e *sem dropout* e *batch normalization*. Registre as curvas de perda de treino e de validação nos dois casos e identifique, em cada um, a época em que o sobreajuste começa a se manifestar.

Exercício 3.5 Estenda a Listagem 3.5 calculando a curva *precision-recall* da CNN (use `precision_recall_curve` sobre as probabilidades previstas). Escolha um limiar que garanta revocação da classe-alvo $\geq 0,95$ e relate a precisão resultante — repetindo, agora para uma rede, a análise de custo do Exercício 2.5.

Exercício 3.6 Considere o problema de *previsão de trajetória* a partir de uma faixa de radar (série temporal de posições). Decida entre uma RNN simples, uma LSTM e um MLP sobre janelas fixas, justifique a escolha e explique, em termos de gradiente evanescente, o risco de tentar capturar dependências longas com uma RNN simples.

Estratégico — extensão criativa

Exercício 3.7 Em um breve texto técnico (até duas páginas), discuta em que circunstâncias

a adoção de um modelo profundo justifica *substituir* uma *baseline* clássica em um sistema crítico de defesa, dado o custo em auditabilidade. Proponha critérios objetivos de decisão — mensuráveis — que uma equipe poderia adotar antes de levar uma rede profunda à produção.

Exercício 3.8 Projete (em diagrama ou pseudocódigo) um *pipeline* de triagem de imagens para apoio ao analista, no espírito do *Project Maven*. Especifique os pontos de *controle humano* na decisão, a métrica operacional adotada e as salvaguardas contra o *viés de automação* (a tendência a confiar acriticamente na saída do modelo). Explique por que a revocação é priorizada.

Exercício 3.9 Escolha um conjunto público de imagens e conduza um estudo comparativo completo entre uma CNN compacta com aumento de dados e uma *baseline* clássica, redigindo um miniartigo de até três páginas que documente: a definição do problema, as escolhas de aumento de dados, as métricas com sua variância e uma recomendação fundamentada — incluindo, de forma explícita, os limites de generalização para imagens operacionais reais e os riscos de *deslocamento de distribuição* (*data shift*).